

Deep Learning-based Intrusion Detection Systems: A Survey

ZHIWEI XU, YUJUAN WU, SHIHENG WANG, JIABAO GAO, TIAN QIU, ZIQI WANG, HAI WAN, and XIBIN ZHAO*, KLISS, BNRist, School of Software, Tsinghua University, China

Intrusion Detection Systems (IDS) have long been a hot topic in the cybersecurity community. In recent years, with the introduction of deep learning (DL) techniques, IDS have made great progress due to their increasing generalizability. The rationale behind this is that by learning the underlying patterns of known system behaviors, IDS detection can be generalized to intrusions that exploit zero-day vulnerabilities. In this survey, we refer to this type of IDS as DL-based IDS (DL-IDS). From the perspective of DL, this survey systematically reviews all the stages of DL-IDS, including data collection, log storage, log parsing, graph summarization, attack detection, and attack investigation. To accommodate current researchers, a section describing the publicly available benchmark datasets is included. This survey further discusses current challenges and potential future research directions, aiming to help researchers understand the basic ideas and visions of DL-IDS research, as well as to motivate their research interests.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; • **Computing methodologies** → **Machine learning**; • **General and reference** → **Surveys and overviews**.

Additional Key Words and Phrases: Intrusion detection systems, deep learning, survey

ACM Reference Format:

Zhiwei Xu, Yujuan Wu, Shiheng Wang, Jiabao Gao, Tian Qiu, Ziqi Wang, Hai Wan, and Xibin Zhao. 2025. Deep Learning-based Intrusion Detection Systems: A Survey. *J. ACM* 1, 1, Article 1 (October 2025), 38 pages.

1 INTRODUCTION

The promising Internet of Everything connects people, processes, data, and things through the Internet [51], bringing convenience and efficiency to the world. Yet its inevitable security vulnerabilities could be exploited by deliberate attackers. With increasingly sophisticated attack methods such as Advanced Persistent Threat (APT), the attackers are in a threatening position to sabotage network systems or steal sensitive data. The detection of intrusions, particularly based on DL, has consequently been a prominent topic in the cybersecurity community.

The automated system for detecting intrusions is known as IDS. The limitations of IDS may result in terrible damage to enterprises. One example is the recent Colonial Pipeline Ransomware Attack [16]. In April 2021, the hacking group DarkSide launched a ransomware attack on Colonial Pipeline, the biggest oil pipeline company in the United States, using an unused VPN account. Due to this attack, 5,500 miles of transportation pipelines were forced to shut down, affecting nearly 45% of the fuel supply on the Eastern Coast. The Colonial Pipeline paid \$4.4 million ransom money, in addition to the theft of over 100 GB of data. If the malware intrusion can be detected in time, the influence of this attack can be greatly mitigated or even eliminated.

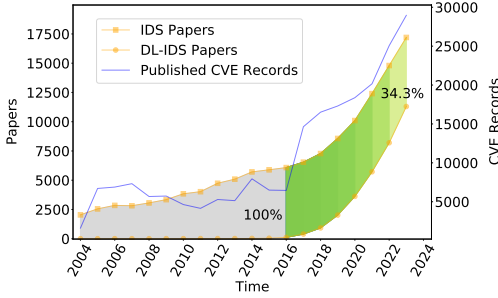
1.1 Tough but Bright Intrusion Detection System

IDS have been increasingly challenged to effectively deal with intrusions for decades. It is noted in Figure 1(a) that the number of CVE¹ records has presented an accelerating uptrend, especially

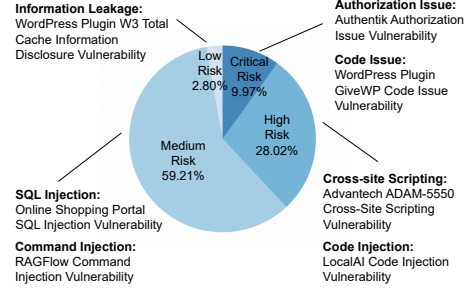
*Xibin Zhao is the corresponding author.

¹Common Vulnerabilities and Exposures (CVE) is a security project for security information sharing and vulnerability management. CVE is a publicly accessible database where each vulnerability has a common name and a unique identifier.

Authors' address: Zhiwei Xu; Yujuan Wu; Shiheng Wang; Jiabao Gao; Tian Qiu; Ziqi Wang; Hai Wan; Xibin Zhao, KLISS, BNRist, School of Software, Tsinghua University, Beijing, China, zxb@tsinghua.edu.cn.



(a) Trend of CVE records and IDS papers.



(b) Category of CNNVD vulnerabilities.

Fig. 1. Recent situation of IDS.

in 2016, which suffered a sharp rise. After 2016, the number of CVE records stays growing at a high speed, reaching around 30,000 in 2024. Besides, according to the CNNVD² report shown in Figure 1(b), we can observe that almost all (i.e., 97.2%) vulnerabilities are medium risk or above, with high and critical risk accounting for 40% of them. The growing number of vulnerabilities and the large percentage of high-risk vulnerabilities both reveal the tough situation faced by IDS.

Nevertheless, an interesting observation from Figure 1(a) is that, against the number of CVE records, DL-IDS papers also started to emerge in 2016 and their amount grew year by year subsequently. We can notably find that the growth trend of DL-IDS papers is nearly the same as that of CVE records. The potential reason can be speculated as DL is an effective way for IDS to cope with their tough situation. Borrowing the strong generalizability from DL techniques, DL-IDS detection can be extended to zero-day intrusions that are almost impossible to detect with the traditional DL-IDS. Some studies [219, 237, 250] demonstrate this speculation. In their experiments, DL-IDS are all reported with an achievement of over 90% detection accuracy while the traditional DL-IDS sometimes only have around 50% detection accuracy.

The IDS future is *not only tough but also bright* with the aid of DL - it is evident that the growth in the number of IDS papers primarily comes from those based on DL techniques. The proportion of DL-IDS papers rises from about 0% in 2016 to a very high 65.7% in 2024. This phenomenon reflects the great interests and visions of the cybersecurity community in DL-IDS. To date, the DL-IDS development has almost reached a decade, and thus, it is time, and also essential, to revisit how DL and IDS interact, identify emerging trends, and guide future research directions.

1.2 Related Surveys and Our Scope

Unfortunately, none of the related surveys in the last decade have systematically investigated DL-IDS. On one hand, some related surveys may only focus on a few parts of DL-IDS, such as log parsers [138, 188, 255], datasets [201], attack modeling [10, 201], and specific DL technique type [17]. On the other hand, while several surveys [21, 83, 96, 105, 127, 128, 140, 150, 162, 163, 270] involve some DL-based approaches, they did not review DL-IDS from the perspective of DL particularly.

Partial Investigation for DL-IDS. The surveys [10, 138, 188, 201, 255] are the typical example papers describing only a few parts of DL-IDS. Among them, Adel et al. [10] mainly studied various

²Chinese National Vulnerability Database (CNNVD) is a Chinese national database that catalogs security vulnerabilities in software and hardware products. CNNVD also provides unique identifiers and descriptions similar to CVE.

techniques and solutions that were tailored to APT attacks, as well as discussed where to make the APT detection framework smart. Scott et al. [138] and Tejaswini et al. [188] detailedly discussed online log parsers and their applications for anomaly detection. Branka et al. [201] review APT datasets and their creation, along with feature engineering in attack modeling. Zhang et al. [255] created an exhaustive taxonomy of system log parsers and empirically analyzed the critical performance and operational features of 17 open-source log parsers. Tristan et al. [17] focused on the applications of graph neural networks (GNNs) to IDS. For DL-IDS, all the above surveys are obviously insufficient to advance research understanding and provide theoretical suggestions.

Different Perspectives from DL-IDS. Another type of existing surveys involved DL-IDS but studied them from the other perspectives [4, 21, 83, 96, 105, 127, 128, 140, 150, 162, 163, 270]. Specifically, the surveys [105, 128] aim to give an elaborate image of IDS and comprehensively explain methods from signature checking to anomaly detection algorithms. Originating from log data, the survey [83] presented a detailed overview of automated log analysis for reliability engineering and introduced three tasks including anomaly detection, failure prediction, and failure diagnosis. In survey [162], Nasir et al. explored the efficacy of swarm intelligence on IDS and highlighted the corresponding challenges in multi-objective IDS problems.

Additionally, data types inspire and contribute significantly to the related surveys, whose categories include host-based IDS (HIDS) [21, 127, 140, 150, 270] and network-based IDS (NIDS) [4, 163]. Bridges et al. [21] focused on IDS leveraging host data for the enterprise network. Martins et al. [150] brought the HIDS concept to the Internet of Things. As a representative form of data in HIDS, the provenance graph [127, 140, 270] and its reduction techniques [96] were also extensively studied in survey literature. In NIDS, Nassar et al. [163] studied the techniques of network intrusion detection, especially those with machine learning (ML). Ahmad et al. [4] further incorporated ML and DL into their NIDS survey and studied the downstream learning methods detailedly.

The above surveys, however, lack investigation and discussion about DL-IDS. DL techniques are only what they cover or involve, rather than the primary focus of their research.

Scope of Our Survey. Our work distinguishes the related surveys by providing a comprehensive literature review of DL-IDS. From the perspective of DL, our survey elaborates on a common workflow of DL-IDS and introduces the corresponding taxonomies of all modules within this workflow. Moreover, our survey discusses the possible challenges and research visions for DL-IDS, which include many DL-related issues that have not yet been studied by the existing surveys.

1.3 Contributions and Organization

In summary, this survey makes the following contributions:

- Realizing that IDS has made significant progress with the aid of DL over the last decade, we present a thorough survey for DL-IDS, formalizing its definition and clarifying its location among other types of IDS.
- We outline the common workflow for DL-IDS, consisting of the data management stage and intrusion detection stage. We further systematically illustrate the research advances in all modules of this workflow and innovatively taxonomize the papers based on DL techniques.
- From the perspective of DL, we discuss the potential challenges and future directions for DL-IDS, especially highlighting the ones unique to DL-IDS for accommodating current researchers.

Survey Structure. Section 2 introduces the survey methodology of this work. Section 3 describes the background knowledge about DL-IDS. Section 4 and Section 5 elaborates the recent research trends on data management stage and intrusion detection stage, respectively. Section 6 illustrates

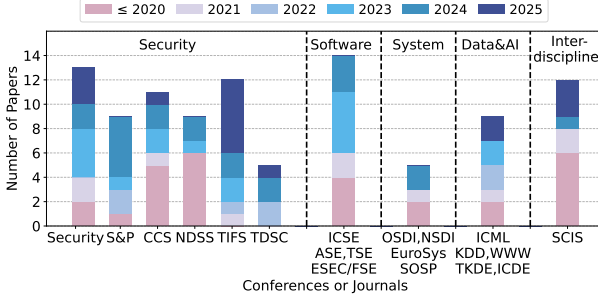


Fig. 2. Source distribution of references.

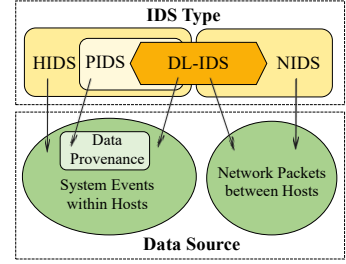


Fig. 3. Types of IDS.

the benchmark datasets and their feature dimensions. Section 7 discusses the visions and challenges for future research. Lastly, the conclusion is presented in Section 8.

2 SURVEY METHODOLOGY

To start our literature review, we selected several popular literature databases, including Web of Science [12], IEEE Xplore [95], and Scopus [50], as the search engine. For search keywords, we determined from generalized terms associated with DL-IDS, such as intrusion detection system, attack investigation, anomaly detection, threat detection, Advanced Persistent Threats, data provenance analysis, forensic analysis, causality analysis, log collection, log compression, log parsing, log storage, and log summarization. Then, we employed Connected Papers [168], a visual tool that assists researchers in finding relevant academic papers, to ensure that we did not overlook the typical related literature. Since the found literature is numerous and rather generalized for the DL-IDS scope, we carefully checked their topics and prioritized only academic papers that are highly related. Finally, all these papers were filtered based on the impact factors of their published journals or academic conferences, leaving us a total of 131 papers.

We identified a few venues that have published many significant papers in the field of DL-IDS, such as Usenix Security, S&P, CCS, NDSS, TIFS, TDSC, ICSE, ASE, ESEC/FSE, TSE, OSDI, NSDI, EuroSys, SOSP, ATC, ICML, KDD, WWW, TKDE, ICDE, and SCIS. We broadly divide them into five categories: security, software, system, data, and interdiscipline. The distribution of these papers with their published years is reported in Figure 2.

3 BACKGROUND

3.1 Intrusion Detection System

3.1.1 Definition of IDS. IDS have long been a central issue in the cybersecurity community, whose research can be traced back to the 1990s [181] or even earlier. According to the existing literature [64, 128, 162, 163, 181, 236], IDS can be defined progressively as follows:

Definition 3.1. (Intrusion Detection System). Intrusion detection system is a software or hardware system to automate the process of *intrusion detection*.

Definition 3.2. (Intrusion Detection). Intrusion detection is the process of monitoring and analyzing the events occurring in a computer or a network for signs of *intrusions*.

Definition 3.3. (Intrusion). Intrusion is the attempt to undermine the confidentiality, integrity, and availability of a computer or a network, or to circumvent its security facilities.

3.1.2 Types of IDS. Generally, IDS can be further categorized into various types based on their data sources [270]. Well-known types include NIDS, HIDS, and Provenance-based IDS (PIDS). Figure 3 depicts IDS types, their data sources, and the location of DL-IDS within those IDS types.

Definition 3.4. (NIDS). NIDS are IDS whose data sources are network traffic between hosts.

NIDS takes network traffic between hosts as its input. It is usually deployed at the edge or key node of the network, allowing it to secure the whole computer system with limited data. Benefiting from the global perception of the whole computer system, NIDS does well in large-scale multi-host intrusions such as Distributed Denial-of-Service (DDoS) attacks. However, NIDS performs poorly in intra-host intrusions and is difficult to analyze intrusions in the form of encrypted network traffic.

Definition 3.5. (HIDS). HIDS are IDS whose data sources are system events within hosts.

HIDS, in contrast, uncovers intrusions through system events of individual hosts. Its data sources include file system changes, system calls, process activities, etc. HIDS can conduct comprehensive detection for a host, and is not affected by encrypted data since the decryption is also performed in the host. Nevertheless, the deployment and maintenance of HIDS is relatively difficult. HIDS should be adapted to hosts of different operating systems and runtime environments. This simultaneously introduces computation overhead for the hosts.

Definition 3.6. (PIDS). PIDS are HIDS whose data sources are data provenance.

Definition 3.7. (Data Provenance). Data provenance refers to the origin and the processes that an event has undergone from its creation to its current state.

PIDS is a subtype of HIDS, particularly referring to HIDS that utilizes data provenance as its data source. Due to analysis in the intact trail of events, PIDS is proven effective in coping with advanced attacks [270]. By performing causality analysis on data provenance, PIDS can significantly reduce false alarms. Yet, data provenance is very expensive to obtain, requiring complicated technical tools for monitoring operating systems, network protocols, and applications.

Definition 3.8. (DL-IDS). DL-IDS are IDS that utilize DL techniques to detect intrusions, whose data sources can be network traffic between hosts, system events within hosts, or their combination.

Unlike the other types of IDS such as NIDS and HIDS are categorized by their data sources, DL-IDS is defined by the techniques used in intrusion detection. As shown in Figure 3, the data source of DL-IDS can be network traffic, system events, or both. Taking advantage of the generalizability of DL techniques, DL-IDS is allowed to handle zero-day attacks precisely and thus become extremely interested in the cybersecurity community recently.

3.2 Common Workflow

Figure 4 depicts the common workflow of DL-IDS. It usually consists of 7 steps: raw data, collection, storage, parsing, summarization, detection, and investigation, which are explained as follows:

- **Raw Data** is unprocessed data for uncovering attack details or benign system behaviors. The raw data analyzed by cyber experts commonly include network traffic and audit logs.
- **Collection** indicates data collection tools for different systems, such as cloud and cross-platforms, which gather valuable raw data to describe important system behavior scenarios.
- **Storage** involves storage and search engines to manage large amounts of collected log data. Log data is labeled with indexes for efficient retrieval.
- **Parsing** is the act of analyzing the stored logs and other useful data. It extracts and organizes the underlying information within the data for subsequent processing.

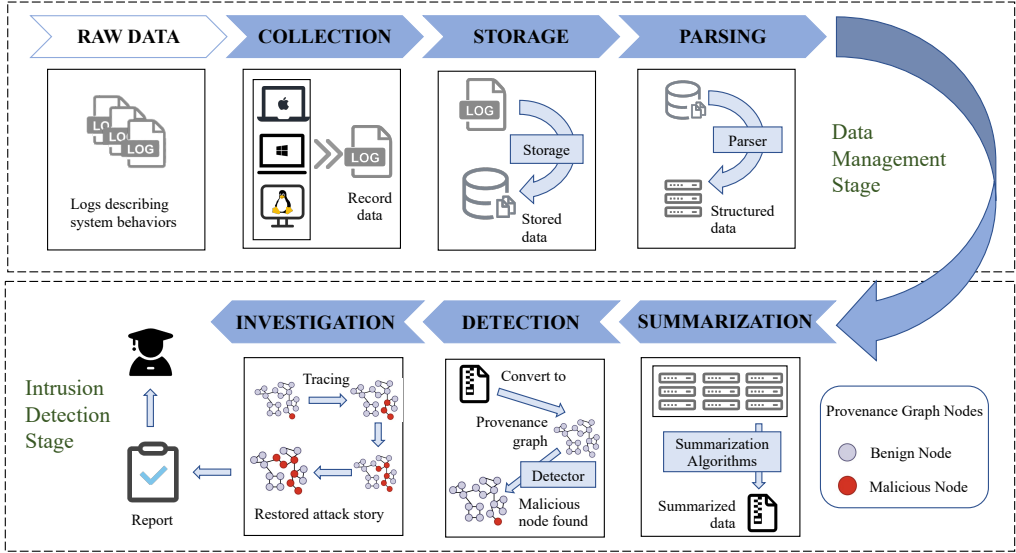


Fig. 4. Common workflow of DL-IDS.

- **Summarization** refers to the operation of summarizing large volumes of parsed data based on its semantics. This reduces storage costs while preserving critical events.
- **Detection** is the process of using detection tools such as models and algorithms to detect anomalies in analyzed data to determine whether the data contains intrusions.
- **Investigation** is the further process of Detection. It reconstructs the entire attack scenarios from the detected malicious data by analyzing the causal relationship between them.

Note that DL-IDS can also be performed in other step orders by skipping some of the steps. For example, log data can be first parsed before storage [135]. Attack investigation can be directly conducted without detection of intrusions [9]. This survey is organized by the common workflow.

4 DATA MANAGEMENT

This section elaborates on the data management stage of DL-IDS, including data collection (Section 4.1), log storage (Section 4.2), and log parsing (Section 4.3).

4.1 Data Collection

The first step of DL-IDS is to collect useful data from raw data. Raw data indicates records that document events, activities, and operations that occur within a system, application, or network (a.k.a., logs), represented by audit logs or application logs within hosts, or network traffic between hosts. By collecting useful logs, DL-IDS is allowed to monitor the health condition and operational status of information systems [141, 255]. Common attributes of logs include timestamp, event type, subject, object, description, etc.

On different platforms, logs possess different formats and organizational structures [21, 127, 255, 270]. To counter this, researchers have created various log collection tools specialized for various systems. For example, in Windows systems, Event Viewer is employed to manage system logs. Yet in Linux systems, log files are usually saved in the `/var/log/` directory. The classification of data collection tools is shown in Table 1, including Windows, Linux, Cloud, and Cross platforms.

Table 1. Log collection tools on different platforms.

Platform Type	Tool	Description
Windows platform	ETW [153]	Providing developers comprehensive event tracing ability
	Panorama [245]	Hardware-level and OS-aware dynamic taint tracking
Linux platform	auditd [68]	Native tools supported by the Linux kernel
	sysdig [106]	Focusing on runtime monitoring and fault troubleshooting
	CamFlow [170]	Self-contained, easily maintainable implementation
	Tracee [210]	Exposing system information as events based on eBPF
	DataTracker [200]	Monitoring unmodified binaries without their source codes
	Inspector [206]	Parallel provenance library that is POSIX-compliant
	AutoLog [94]	Analyzing programs so no need to run them
Cloud platform	eAudit [193]	Fast, scalable and easily deployable data collection tools
	K8S tools [27, 87]	Adapting to cloud scenarios to meet enterprise needs
	saBPF [129]	An extension tool of eBPF for containers in cloud computing
Cross platform	ISDC [158]	Eliminating overheads on in-network resources
	DTrace [66]	Real-time tracing framework that supports many platforms
	SPADE [61]	Novel provenance kernel for cross-platform logging

4.1.1 Windows Platform Tools. Event Tracing for Windows (ETW) [153] is a powerful event tracing mechanism provided by Microsoft. It consists of three components: providers, controllers, and consumers. ETW instruments applications to provide kernel event logging and allows developers to start and stop event tracing sessions momentarily. Panorama [245] exploits hardware-level and OS-aware dynamic taint tracking to collect logs. Moreover, it develops a series of automated tests to detect malware based on several kinds of anomalous behaviors.

4.1.2 Linux Platform Tools. auditd [68] is a native collection tool supported by the Linux kernel, which is responsible for writing audit logs to disk and monitoring a variety of auditable events such as system calls, file accesses, and modifications. sysdig [106] relies on the kernel module to achieve monitoring and data collection of the system. sysdig focuses on system runtime monitoring and fault troubleshooting, which is also widely used in containers and cloud-native environments. CamFlow [170] designs a self-contained, easily maintainable implementation of whole-system provenance based on Linux Security Module, NetFilter, and other kernel facilities. Furthermore, it provides a mechanism to adapt the captured data provenance to applications and can be integrated across distributed systems. Tracee [210] takes advantage of the extended Berkeley Packet Filter (eBPF) framework to observe systems efficiently. It uses eBPF to tap into systems and expose that information as events. DataTracker [200] is an open-source data provenance collection tool using dynamic instrumentation. It is able to identify data provenance relations of unmodified binaries without access to or knowledge of the source codes. Inspector [206] is a Portable Operating System Interface (POSIX)-compliant data provenance library for shared-memory multi-threaded applications. It is implemented as a parallel provenance algorithm on a concurrent provenance graph. AutoLog [94] generates runtime log sequences by analyzing source codes and does not need to execute any programs. It can efficiently produce log datasets (e.g., over 10,000 messages/min on Java projects) and has the flexibility to adapt to several scenarios. eAudit [193] is a scalable and easily deployable data collection tools. eAudit relies on the eBPF framework built into recent Linux versions, making it work out of the box on most of the Linux distributions.

4.1.3 Cloud Platform Tools. Although some collection tools in Windows and Linux platforms such as auditd [68], sysdig [106], and Tracee [210] can be applied in cloud computing environment, cloud-native scenarios introduce different challenges compared with Windows or Linux platforms. First,

there are many different types of components such as containers, microservices, and Kubernetes (K8S) clusters in cloud platforms, each of which generates its own logs with varying formats and contents. Additionally, components are basically characterized by dynamic expansion and contraction, making it hard to capture complete log data. To address them, Chen et al. [27] design a cloud log collection architecture on the basis of K8S, which is a central platform based on cloud-native technology. Josef et al. [87] propose a log collection and analysis tool operated as Software as a Service (SaaS) in the cloud environment in K8S technology, aiming to provide comprehensive logs across all microservices. saBPF [129] is an extension tool of eBPF, aiming to deploy fully-configurable, high-fidelity, system-level audit mechanisms at the granularity of containers. saBPF is further developed with proof-of-concept IDS and access control mechanism to demonstrate its practicability. ISDC [158] is designed to eliminate the bottleneck between network infrastructure (where data is generated) and security application servers (where data is consumed), which prioritizes specific flows to effectively optimize resource consumption.

4.1.4 Cross-platform Tools. To effectively detect intrusions, an intuitive idea is to incorporate log data from various platforms to obtain a global view of the running system. DTrace [66] is a real-time dynamic tracing framework for troubleshooting kernel and application problems on production systems. It supports many platforms, including Linux, Windows, Solaris, macOS, FreeBSD, NetBSD, etc. Support for Provenance Auditing in Distributed Environments (SPADE) [61] develops a novel provenance kernel that mediates between the producers and consumers of provenance information, and handles the persistent storage of records. It supports heterogeneous aggregating for system-level data provenance for data analysis across multiple platforms.

4.2 Log Storage

The subsequent step of log collection is to store these logs [11, 40]. We will introduce two essential components for data storage: log storage systems and compression algorithms for these systems.

4.2.1 Log Storage Systems. The two most commonly used log storage systems are ELK [5] and Loki [15]. ELK is a powerful log management solution consisting of three open-source software components: Elasticsearch [48], Logstash [47], and Kibana [49]. Elasticsearch [48] is the leading distributed, RESTful search and analytics data engine designed with speed and scalability. Logstash [47] is a server-side data preprocessing pipeline to collect and integrate data from multiple sources. Kibana [49] is a data analytics and visualization platform at both speed and scale. ELK is powerful enough to be applied in enterprise scenarios, however, its performance comes at a price. ELK sacrifices ease of configuration and installation, and may simultaneously introduce severe runtime overhead for its hosts. In contrast, Loki [15] is a lightweight logging system with low resource overhead developed by Grafana Labs. It is designed with simple operations and efficient storage. Instead of indexing everything of data like ELK does, Loki mainly creates indices grounded in log labels. Moreover, Loki is well suited for open-source monitoring and visualization tools such as Prometheus [174] and Grafana [112]. Integrating these two tools enables Loki to construct a complete monitoring and log analysis platform for information systems.

4.2.2 Log Compression Algorithms. Logs are generated quickly and require significant memory usage. For example, it is measured that a browser can produce about 10 GB of log data each day [40]. Such oversize data should be compressed before storage. Log compression algorithms can be categorized into two types: general-purpose algorithms and those specifically adapted to log data.

General Compression Algorithms. General compression algorithms refer to algorithms to reduce the size of data (e.g., log data) by handling token-level or byte-level duplicates in the data. General compression algorithms can be classified into three categories based on their principles [242]:

Table 2. Well-acknowledged general compression algorithms for log data.

Type	Well-acknoledged compression algorithm
Dictionary-based	LZ77 in <i>gzip</i> [55], LZMA in <i>7zip_lzma</i> [171], and LZSS in <i>quickLZ</i> [177]
Sorting-based	BWT in <i>bzip2</i> [194] and ST in <i>gzip</i> [190]
Statistical-based	PPMD in <i>7zip_ppmd</i> and DMC in <i>ocamyd</i> [191]

- Dictionary-based Compression: It records repeated data as keys and replaces these data with their corresponding keys.
- Sorting-based Compression: It sorts data to enable strategies that require ordering features.
- Statistical-based Compression: It exploits statistical techniques to learn and predict the possible next token for existing tokens. The data is thus compressed as a statistical model.

Table 2 presents representative algorithms of the above three types. Due to the indeterminacy of statistical techniques, statistical-based compression algorithms may introduce losses in compression. Yet the other two types of algorithms are generally lossless. By validating 9 log files and 2 natural language files, a study [242] shows that some general compression algorithms can achieve high compression ratios for log data and log data is even easier to compress than natural language data.

Tailored Compression Algorithms. Different from natural language data, log data usually has specific structures and formal expressions that help further compression. Yao et al. [243] propose LogBlock, which obtains small log blocks before compression and then uses a generic compressor to compress logs. Liu et al. [135] propose Logzip, which employs clustering algorithms to iteratively extract templates from raw logs and then obtain coherent intermediate representations for compressing logs. Rodrigues et al. [186] propose the lossless compression tool CLP, aiming to quickly retrieve log data while meeting compression requirements. CLP proposes to combine domain-specific compression and search with a generic lightweight compression algorithm. Li et al. [123] conduct empirical research on log data and propose LogShrink to overcome their observed limitations by leveraging the commonality and variability of log data. LogBlock [243] is designed to help existing jobs perform better. It reduces duplicate logs by preprocessing log headers and rearranging log contents, thereby improving the compression ratio of log files. LogReducer [247] is a framework that combines log hotspot identification and online dynamic log filtering. Its non-intrusive design significantly reduces log storage and runtime overhead. μ Slope [217] is a compression and search method for semi-structured log data. It achieves efficient storage and query performance through data segmentation, pattern extraction, and index-free design. Denum [249] significantly improves log compression rates by optimizing the compression of digital tokens in logs. It is an efficient log compression tool suitable for scenarios where you need to save storage space or transmission bandwidth.

4.3 Log Parsing

Log data often originates from multiple different devices such as terminals, sensors, and network devices. To analyze it, log parsers are employed to format them into structured and unified ones. Log parsing is usually executed by data classification and template extraction. Data classification is to classify log data into several groups. Each group constitutes a template for extracting features from log data and constructing the structured logs. As shown in Figure 5, the existing log parsers can be taxonomized into 3 categories: clustering-based, pattern-based, and heuristic-based parsers.

4.3.1 Clustering-based Parsing. Clustering-based parsers classify data using clustering algorithms for log parsing. Xiao et al. [226] propose LPV, which employs a hierarchical clustering algorithm

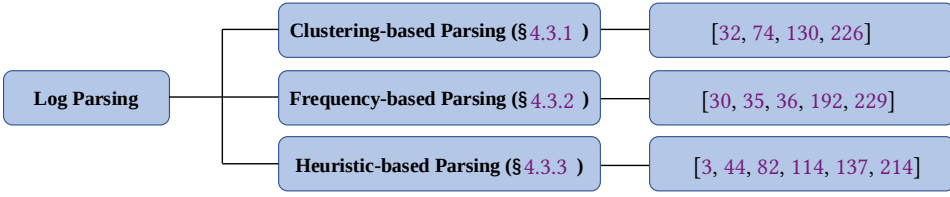


Fig. 5. Taxonomy of data parsing.

to incrementally group logs based on Euclidean distance. Hamooni et al. [74] present a rapid log pattern recognition approach named LogMine. It is implemented in the map-reduce framework for distributed platforms to process millions of log messages in seconds. LogCluster [130] reduces the number of logs that need to be manually checked and improves the accuracy of problem identification through log clustering and the use of knowledge bases. METING [32] provides a robust and efficient log parsing method through frequent n-gram mining and flexible log grouping strategy, which can effectively process various types of log data.

4.3.2 Frequency-based Parsing. Frequency-based parsers discover patterns that exceed the frequency threshold and employ the mined patterns to parse logs. Sedki et al. [192] propose the log parsing tool ULP, which combines string matching and local frequency analysis to efficiently parse large log files. Dai et al. [35] propose Logram, which utilizes an n-gram dictionary for log parsing. For n-grams with a frequency below the threshold, Logram recursively converts to (n-1)-grams until a list of uncommon 2-grams is obtained. To mitigate the parameter sensitivity issue in log parsers, Dai et al. [36] further proposed an entropy-based log parser PILAR, which balances parsing accuracy and efficiency. Xu et al. [229] propose a hybrid log parsing model called Hue, which performs parsing through user-adaptive methods. Prefix-Graph [30] is an efficient, adaptive, and universal log parsing method that can stably extract log templates without relying on domain knowledge and manual parameter tuning.

4.3.3 Heuristic-based Parsing. Heuristic-based parsers rely on empirical knowledge to classify log data. He et al. [82] propose the online log parsing method Drain, which employs a depth-fixed parsing tree to group the original logs and encodes them using specially designed parsing rules. Le et al. [114] propose to use a hint-based few-sample learning algorithm, LogPPT, to capture log template patterns. Utilizing new prompt tuning methods and an adaptive random sampling algorithm, LogPPT performs well on multiple public datasets. Liu et al. [137] propose the UniParser parser to address the issue of difficult processing of heterogeneous logs, using the Token Encoder and Context Encoder modules to learn log context features. Spell [44] is an efficient streaming log parsing method that can dynamically extract log patterns in online processing and significantly improve processing efficiency through pre-filtering steps. Logan [3] achieves efficient and scalable log parsing through distributed processing, LCS matching, dynamic matching tolerance, and periodic merging. USTEP [214] is an online log parsing method based on an evolutionary tree structure that can discover and encode new parsing rules. It achieves constant parsing time and can efficiently parse raw log messages in a streaming manner.

5 INTRUSION DETECTION

The intrusion detection stage uncovers intrusions relying on the semantic-level information. This section classifies and summarizes the mainstream graph summarization (Section 5.1), attack detection (Section 5.2), and attack investigation (Section 5.3).

Table 3. Overview of graph summarization approaches.

Mode	Approach	Release	Baseline	Requirement
Offline	ProvCompress [228]	2011	No Summarization	None
	BEEP [115]	2013	No Summarization	Instrumentation
	LogGC [116]	2013	BEEP + No Summarization	Instrumentation
	CPR + PCAR [234]	2016	No Summarization	None
	FD + SD [89]	2018	CPR + PCAR	None
	LogApprox [152]	2020	GC + CPR + DPR	None
	TeRed [122]	2025	LogGC + CPR + PCAR + F-DPR + NodeMerge	None
Online	ProTracer [143]	2016	BEEP + No Summarization	Instrumentation
	NodeMerge [205]	2018	No Summarization	None
	Winnower [77]	2018	No Summarization	None
	GS + SS [267]	2021	FD + SD	None
	SEAL [53]	2021	FD	None
	FAuST [97]	2022	CPR + DPR	None
	AudiTrim [202]	2024	CPR + GS + F-DPR	None

5.1 Graph Summarization

It is illustrated that stealthy malware will inevitably interact with the underlying OS and be captured by provenance monitoring systems [216], which is the reason why PIDS (a form of DL-IDS) has worked and flourished recently. Log data generated from provenance monitoring systems is referred to as data provenance as mentioned. Offering advantages in high precision, data provenance sacrifices memory performance to record all trails of events from their creations to their current states, even some of which are trivial. Unlike network traffic and application logs, data provenance is fine-grained, detailed, and rich in semantics. As a result, the token-level or byte-level log storage systems (Section 4.2.1) and log compression algorithms (Section 4.2.2) are insufficient to handle the memory efficiency of data provenance due to the absence of semantic-level information.

To this end, graph summarization is investigated to further reduce the size of log data semantically. In graph summarization, data provenance is transformed into a provenance graph, of which the causal relations are utilized to build the semantic understanding of system activities. Referring to the definition of data provenance (Definition 3.7), provenance graph is defined as follows:

Definition 5.1. (Provenance Graph). Provenance graph is a representation of a collection of data provenance with causal relations. It is a directed acyclic graph $G = \langle V, E \rangle$ where nodes V are system entities and edges E are system events.

Provenance graphs allow graph summarization approaches to reduce the size of log data by confidently removing irrelevant events, aggregating similar events, gathering similar execution entities, etc. This categorizes them as a type of lossy reduction, yet the aforementioned log storage and compression are usually lossless (except for statistical-based log compression). We note that some surveys (e.g., [96, 270]) may interchangeably use graph summarization and log compression to identify the approaches that reduce the size of log data. In this work, we explicitly distinguish them and refer to the lossless reduction as compression and the opposite one as summarization. Table 3 presents the overview of graph summarization approaches. We classify them into two categories: offline graph summarization and online graph summarization.

5.1.1 Offline Graph Summarization. Offline graph summarization requires historical log data to provide global knowledge, which extracts log data from persistent storage, summarizes the data, and pushes back the summarized data to the persistent storage. In 2011, Xie et al. [228] take inspiration from web graphs to summarize provenance graphs. They argue that provenance

graphs have similar organizational structure and characteristics to web graphs, such as locality, similarity, and consecutiveness. BEEP [115] is developed based on the fact that a long-running execution can be partitioned into individual units. BEEP reverse engineers application binaries and instructions to perform selective logging for unit boundaries and unit dependencies. LogGC [116] is a summarized audit log system that can be invoked at any time during the system execution. Xu et al. [234] propose an aggregation algorithm PCR that preserves event dependencies during log data reduction. They further propose an algorithm named PCAR that utilizes domain knowledge to conduct graph summarization. Hossain et al. [89] propose two dependency-preserving graph summarization approaches, FD and SD. FD is allowed to keep backward and forward forensic analysis results. SD preserves the results of common forensic analysis, which runs backward to find the entry points of intrusions and then runs forward from these points to unveil their impacts. LogApprox [152] aims to summarize the most space-intensive events found in logs, namely file I/O activity, which can account for up to 90% of the log content. TeRed [122] employs unit tests to learn the system's normal behavior patterns for reducing provenance graphs, allowing it not to impact attack detection and investigation.

5.1.2 Online Graph Summarization. Online graph summarization performs real-time summarization for continually coming provenance graphs, rather than dealing with a static provenance graph. ProTracer [143] alternates between system event logging and unit-level taint propagation. It has a lightweight kernel module and user space daemon for concurrent, out-of-order event processing. NodeMerge [205] is a template-based graph summarization system for online event storage. It can directly work on the system-dependent provenance streams and compress data provenance via read-only file access patterns. Winnower [77] is an extensible audit-based cluster monitoring system. For tasks replicated across nodes in distributed applications, it can define a model over audit logs to concisely summarize the behaviors of multiple nodes, thus eliminating the necessity of transmitting redundant audit records to the central monitoring node. The approach proposed by Zhu et al. [267] includes two real-time graph summarization strategies. The first strategy maintains global semantics, which identifies and removes redundant events that do not affect global dependencies. The second strategy is based on suspicious semantics. SEAL [53] is a novel graph summarization approach for causal analysis. Based on information-theoretic observations of system event data, it achieves lossless compression and supports real-time historical event retrieval. FAuST [97] is a logging daemon that performs transparent and modular graph summarization directly on system endpoints. FAuST consists of modular parsers that parse different audit log formats to create a unified in-memory provenance graph representation. AudiTrim [202] is an efficient graph summarization approach that reduces log sizes without impacting user experiences, which allows adaptable deployment on different operating systems.

5.2 Attack Detection

Attack detection is located at the central position of DL-IDS. The objective of attack detection is to accurately identify malicious system events in log data while minimizing false alarms of normal system behaviors. Based on the types of log data, we categorize the attack detection approaches into audit log-based, application log-based, network traffic-based, and hybrid log-based detectors.

The overview and taxonomy of attack detection approaches are presented in Table 4. We note that recent years have also published many other academic papers for attack detection [25, 46, 78, 119, 156, 218, 224, 227, 248]. Yet these papers are slightly related to DL-IDS, which are thus excluded in our survey for conciseness.

Table 4. Overview and taxonomy of attack detection approaches.

Data Type	Taxonomy	Approach	Release Time	Base Model	Detection Style	Detection Granularity
Audit Log	Traditional Learning	StreamSpot [145]	2018	K-Medoids	Online	Subgraph
		Unicorn [76]	2020	K-Medoids	Online	Node, Subgraph
		DistDet [42]	2023	HST	Online	Subgraph
		Velox [18]	2025	FCN	Online	Node
	Graph Neural Network	ShadeWatcher [250]	2022	TransR	Offline	Node
		threaTrace [219]	2022	GraphSAGE	Online	Node
		ProGrapher [237]	2023	graph2vec	Online	Subgraph
		MAGIC [99]	2024	GAT	Online	Node, Subgraph
		Flash [182]	2024	GraphSAGE	Online	Node
		R-caid [65]	2024	GNN	Offline	Node
		Argus [230]	2024	MPNN, GRU	-	Node
		TAPAS [252]	2025	LSTM-GRU	Online	Task
Application Log	Traditional Learning	Wei et al. [231]	2009	PCA, TF-IDF	-	Log Entry
		Bodik et al. [19]	2010	Logistic Regression	Online	Log Entry
		AMOD [43]	2018	SVM HYBRID	Online	Log Entry
	Sequence Neural Network	DeepLog [45]	2017	LSTM	Online	Log Entry
		LogRobust [257]	2019	Attention LSTM	-	Log Entry
		LogAnomaly [151]	2019	template2vec, LSTM	Online	Log Entry
		LogC [246]	2020	LSTM	Online	Log Entry
		NeuralLog [113]	2021	BERT	-	Log Entry
		PLELog [238]	2021	Attention GRU	Online	Log Entry
		SpikeLog [175]	2023	DSNN	-	Log Entry
		LogCraft [254]	2024	Meta Learning	-	Log Entry
		Tweezers[33]	2024	GATv2, BERTweet	Online	Log Entry
		LogSer [23]	2024	BERT	Online	Log Entry
		LogDLR[265]	2025	Transformer, SBERT	Online	Log Entry
Traffic Log	Traditional Learning	NetPro [121]	2017	Merkle Hash Tree	Online	Route
		CATH [72]	2019	Cusp Model	Online	Flow
		Whisper [56]	2021	K-Means	-	Host
		SigML++ [211]	2023	ANN	-	Encrypted Log
		OADS [253]	2023	Isolation Forest	Online	Packet
		LtRFT [204]	2023	LambdaMART	Offline	Packet
		AGC [225]	2025	Clustering	-	Packet
	Graph and Sequence Neural Network	Kitsune [159]	2018	AutoEncoder	Online	Packet
		MT-FlowFormer [260]	2022	Transformer	-	Flow
		I ² RNN [199]	2022	I ² RNN	-	Packet
		ERNN [262]	2022	ERNN	-	Flow
		Euler [108]	2023	GNN, RNN	-	Flow
		pVoxel [58]	2023	-	-	Packet, Flow
		NetVigil [91]	2024	E-GraphSage	-	Flow
		Exosphere [57]	2024	CNN	-	Packet
		DFNet [263]	2024	DFNet	-	Packet
		RFH-HELAD [264]	2024	RPGAN, Deep kNN	-	Packet
		ReTrial [259]	2024	Bayesian Inference	Online	Flow
		HEN [221]	2024	AE-LSTM	-	Packet, Flow
		TCG-IDS [222]	2025	TGN	Online	Flow
		A-NIDS[251]	2025	Stacked CTGAN	Online	Flow
		GTAE-IDS[62]	2025	Graph Transformer	Online	Packet, Flow
Hybrid	Hybrid	OWAD [75]	2024	Autoencoder	Online	Hybrid
		FG-CIBGC [165]	2025	DisenGCN, ICL	-	Hybrid

5.2.1 Audit Log-based Detectors. Audit logs are collected from hosts and thus detectors based on them are basically referred to as HIDS. Audit logs provide fine-grained information through provenance graphs to depict system behaviors. Depending on the learning techniques, audit log-based detectors can be further classified as traditional learning and graph neural network.

Traditional Learning. Traditional learning-based detectors refer to those that utilize naive machine learning techniques. StreamSpot [145] is a clustering-based anomaly detection that tackles challenges in heterogeneity and streaming nature. Unicorn [76] is a real-time intrusion detector that efficiently constructs a streaming histogram to represent the history of system executions. The counting results within the histogram are updated immediately if new edges (or events) occur. DistDet [42] is a distributed detection system that builds host models in the client side, filters false alarms based on their semantics, and derives global models to complement the host models. Velox [18] derives from Orthrus and replaces the complex TGN-based encoder with a simple fully-connected network (FCN), leading to a lightweight and efficient neural network.

Graph Neural Network. GNN is demonstrated to do well in processing provenance graphs [99, 182, 219, 237, 250]. ProGrapher [237] extracts temporal-ordered provenance graph snapshots from the ingested logs, and applies whole graph embedding and sequence-based learning to capture rich structural properties of them. The key GNN technique leveraged by ProGrapher is graph2vec. ShadeWatcher [250] is a recommendation-guided intrusion detector using provenance graphs. It borrows the recommendation concepts of user-item interactions into security concepts of system entity interactions and analyzes cyber threats in an automated and adaptive manner. threaTrace [219] emerges as an online approach dedicated to detecting host-based threats at the node level. Its GNN model is a tailored GraphSAGE [73] for learning rich contextual information in provenance graphs. MAGIC [99] leverages Graph Attention Network (GAT) [213] as its graph representation module. MAGIC employs masked graph representation learning to incorporate the capability of pre-training. It can adapt to concept drift with minimal computational overhead, making it applicable to real-world online APT detection. Flash [182] is a comprehensive and scalable approach on data provenance graphs to overcome the limitations in accuracy, practicality, and scalability. Flash incorporates a novel adaptation of a GNN-based contextual encoder to encode both local and global graph structures into node embeddings efficiently. R-caid [65] first incorporates root cause analysis into PIDS. Before training GNNs, R-caid links nodes to their root causes to build a new graph, intending to prevent it from mimicry and evasion attacks. Argus [230] finds the performance of the prior IDS is questionable on large scale. It thus devises a form of discrete temporal graph and uses encoder-decoder unsupervised learning to detect different types of attacks. TAPAS [252] leverages a stacked LSTM-GRU model and a task-guided segmentation algorithm to reduce the spatiotemporal dimensions of APT detection, achieving efficient, low-cost, and accurate detection. In addition to the aforementioned detectors, recent researchers have developed numerous useful tools for better understanding audit logs, such as data visualization analysis tool [133] and counterfactual-driven attack explanation generator [223].

5.2.2 Application Log-based Detectors. Application logs are generated from the installed binaries. Generally, application logs are in the form of natural language text, namely sequence data. It is thus common to introduce sequence-based DL techniques into application log-based DL-IDS.

Traditional Learning. For traditional learning, Wei et al. [231] propose a general methodology to mine rich semantic information in console logs to detect large-scale system problems. Bodik et al. [19] leverage a logistic regression model on a new and efficient representation of a datacenter's state called fingerprint to detect previously seen performance crises in that datacenter. AMOD [43] uses the SVM HYBRID strategy to filter query annotations from web request logs and then

update the stacked generalization detection model to efficiently detect web code injection attacks and obtain malicious queries to update the web application firewall (WAF) library.

Sequence Neural Network. Due to the similarity between application logs and natural language texts, sequence neural networks such as Recurrent Neural Network [86] and Transformer [39, 212] are widely employed. DeepLog [45] employs LSTM to model system logs as natural language sequences. It is able to automatically learn benign log patterns and detect anomalies when there is a deviation between log patterns and the trained model. LogRobust [257] finds previous methods do not work well under the close-world assumption and utilizes an attention-based LSTM model to handle unstable log events and sequences. LogAnomaly [151] identifies previous studies tend to cause false alarms by using indexes rather than semantics of log templates. Empowered by a novel, simple yet effective method termed template2vec, LogAnomaly is proven to successfully detect both sequential and quantitative log anomalies simultaneously. LogC [246] is a new log-based anomaly detection approach with component-aware analysis. It feeds both log template sequences and component sequences to train a combined LSTM model for detecting anomalous logs. NeuralLog [113] targets the performance caused by log parsing errors such as out-of-vocabulary words and semantic misunderstandings and employ BERT to perform neural representation. PLELog [238] is a semi-supervised anomaly detection approach that can get rid of time-consuming manual labeling and incorporate the knowledge on historical anomalies. SpikeLog [175] adopts a weakly supervised approach to train an anomaly score model, with the objective of handling a more reasonable premise scenario where a large number of logs are unlabeled. LogCraft [254] is an end-to-end unsupervised log anomaly detection framework based on automated machine learning, which mitigates the cost of understanding datasets and makes multiple attempts for building algorithms. Tweezers [33] uses a large language model to identify entities and build a relationship graph, and generates embeddings through graph attention network optimization to achieve security incident detection. LogSer [23] parses logs by preprocessing parameters, splitting logs, tree parsing, and template merging. It then inputs relevant embeddings into BERT training to detect anomalies, generate reports, and perform incremental updates. LogDLR [265] uses SBERT embeddings and a Transformer autoencoder with domain adversarial training to learn domain-invariant features, detecting anomalies via reconstruction error.

5.2.3 Network Traffic-based Detectors. Network traffic comes from communications between hosts across a computer network. It is ruled by network protocols such as Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP) and can be utilized for intrusion detection. Basically, network traffic-based detectors are termed NIDS.

Traditional Learning. Given the fact that network traffic is usually encrypted for secure communications, feature engineering-guided machine learning is widely applied in NIDS. NetPro [121] employs traceability reasoning with Merkle Hash Trees and digital signatures to detect direct and indirect MANET routing attacks while preserving node privacy, and outputs a traceability graph to identify malicious nodes and behaviors. CATH [72] is a catastrophe-theory-based approach for DoS detection in software-defined networks (SDNs), which leverages the selection, normalization, and fusion of statistical flow attributes to model network states. Whisper [56] pays attention to both high accuracy and high throughput by utilizing frequency domain features. SigML++ [211] is an extension of SigML for supervised anomaly detection approach. SigML++ employs Fully Homomorphic Encryption and Artificial Neural Network (ANN) for detection, resulting in execution without decrypting the logs. OADSD [253] achieves task independently and has the ability of adapting to the environment over SD-WAN by using On-demand Evolving Isolation Forest. LtRFT [204] innovatively introduces Learning-To-Rank scheme for mitigating the low-rate DDoS

attacks targeted at flow tables. AGC [225] maps the original data into the embedding space through embedding learning to obtain more representative anchor points, thus achieving fine-grained classification of low-quality label data.

Graph and Sequence Neural Network. In network traffic, packets consist of various contents and their flows can be represented as graphs. As a result, both graph neural network and sequence neural network are adopted in NIDS. Kitsune [159] is a plug and play NIDS that is allowed to detect attacks efficiently on the local network without supervision. It alleviates the problem that network gateways and router devices simply do not have the memory or processing power. MT-FlowFormer [260] is a semi-supervised framework to mitigate the lack of a mechanism for modeling correlations between flows and the requirement of a large volume of manually labeled data. I²RNN [199] is an incremental and interpretable RNN for encrypted traffic classification, which can be efficiently adapted for incremental traffic types. ERNN [262] represents error-resilient RNN, which is a robust and end-to-end RNN model specially designed against network-induced phenomena. Euler [108] accelerates the most memory-intensive part, message-passing stage within GNN, with several concurrently-executed replicated GNNs. pVoxel [58] is an unsupervised method that proposes to leverage point cloud analysis to reduce false positives for the previous NIDS such as Whisper and Kitsune without requiring any prior knowledge on the alarms. NetVigil [91] is specially designed for east-west traffic within data center networks. It utilizes E-GraphSage and contrastive learning techniques to strengthen its resilience. Exosphere [57] detects flooding attacks by analyzing packet length patterns, without investigating any information in encrypted packets. DFNet [263] is a DDoS prevention paradigm denoted by preference-driven and in-network enforced shaping. RFH-HELAD [264] consists of a K classification model based on a deep neural network and a $K + 1$ classification combining GAN and Deep kNN for detecting anomalies in network traffic. ReTrial [259] employs an improved graph attention network with Bayesian and EM algorithms to iteratively correct misleading links, enabling robust detection of encrypted malicious traffic. HEN [221] uses SMOTE to enhance data, trains LightGBM, generates explanations via SHAP, trains AE-LSTM to reconstruct SHAP values, sets a threshold from training errors, and marks test traffic with excess errors as attacks for intrusion detection. TCG-IDS [222] is the first self-supervised temporal contrastive GNN for network intrusion detection, capturing spatiotemporal traffic dependencies with high accuracy and low false alarms. A-NIDS [251] uses a shallow fully connected network for real-time detection and a Stacked CTGAN generator to address catastrophic forgetting and old data storage costs. GTAE-IDS [62] uses a graph autoencoder with a Transformer encoder and DNN decoder to learn benign traffic, enabling label-free, near-real-time intrusion detection and new attack identification.

5.2.4 Hybrid Log-based Detectors. Based on the above discussions, a natural idea is to combine various types of log data for improving detection capability. OWAD [75] is a general framework to detect, explain, and adapt to normality shifts in practice. OWAD is validated to be effective in various detection granularity, covering provenance graphs, application logs, and network packets. FG-CIBGC [165] mines syncretic semantics in multi-source logs including audit logs, application logs, and network traffic using LLM under in-context learning, which generates behavior graphs for comprehensive analysis.

5.3 Attack Investigation

Except for identifying individual intrusive nodes, IDS are supposed to detect the full story of intrusions (a.k.a., attack scenario graphs). This process is referred to as attack investigation, which can be done by directly detecting attack scenario graphs [216], or analyzing the causal relations between compromised nodes progressively to construct attack scenario graphs [9, 41, 100, 232]. The attack scenario graphs are defined with scenario graphs as follows:

Table 5. Overview of attack investigation approaches.

Taxonomy	Approach	Release Time	Audit Log	Application Log	Base Model	Starting Node	Investigation Granularity
Traditional Learning	ProvDetector [216]	2020	✓	✗	doc2vec	✗	Path
	BehaviorBaseline [269]	2025	✓	✗	FastText	✗	Path
Sequence Neural Network	ATLAS [9]	2021	✓	✓	LSTM	✓	Graph
	LogTracer [166]	2022	✓	✓	DeepLog	✓	Path
	ConLBS [118]	2023	✓	✗	Transformer	✓	Graph
	AirTag [41]	2023	✓	✓	BERT	✗	Graph
Graph Neural Network	Liu et al. [134]	2022	✓	✗	struc2vec	✓	Graph
	Karios [29]	2023	✓	✓	GNN	✗	Graph
	TREC [139]	2024	✓	✗	GNN	✗	Graph
	Orthrus [100]	2025	✓	✗	UniMP	✗	Path
	Slot [176]	2025	✓	✗	GNN	✗	Graph
	FeCoGraph [146]	2025	✗	✗	GCN	✗	Graph

Definition 5.2. (Scenario Graph). Scenario graph is a subgraph of its given provenance graph, which is constructed by the nodes and edges causally dependent on *nodes of interest*.

Definition 5.3. (Attack Scenario Graph). Attack scenario graph is a scenario graph where its nodes of interest are compromised nodes.

In the past, attack investigation is conducted by forward analysis and backward analysis [88]. Forward analysis discovers the influence that nodes of interest will cause and backward analysis traces back how nodes of interest are generated. Benefiting from DL techniques, both forward and backward analysis can be achieved by learning patterns of attack scenario graphs. Furthermore, visual analytics techniques have been widely used to assist security analysts in understanding the causal chain of intrusions [256, 261]. Table 5 summarizes the overview of attack investigation approaches. Similar to Section 5.2, we exclude papers [6, 52, 60, 80, 88, 98, 111, 120, 142, 157, 218, 239, 268] slightly relevant to DL for conciseness.

Traditional Learning. Unlike detecting intrusive nodes, attack scenario graphs are complicated and thus are hard to handle by traditional learning methods. ProvDetector [216] utilizes doc2vec to learn the embedding representation of paths in the provenance graph. Then a density-based detection is deployed to detect abnormal causal paths in the provenance graph. BehaviorBaseline [269] presents a novel learning-based anomaly detection method for large-scale provenance graphs. It incorporates dynamic graph processing with adaptive encoding and a tag-propagation framework for real-time detection.

Sequence Neural Network. Log data is in the form of natural language text or is allowed to be transformed into sequences of events, which facilitates the introduction of sequence neural networks. ATLAS [9] is a framework to construct end-to-end attack stories from readily available audit logs, which employs a novel combination of causal analysis and natural language processing. ATLAS exploits LSTM to automatically learn the pattern difference between attack and non-attack sequences. LogTracer [166] is an efficient anomaly tracing framework that combines data provenance and system log detection together. An outlier function with an abnormal decay rate is introduced to improve the accuracy. ConLBS [118] combines a contrastive learning framework and multilayer Transformer network for behavior sequence classification. AirTag [41] employs unsupervised learning to train BERT directly from log texts rather than relying on provenance graphs. AirTag constructs attack scenario graphs by integrating the detected victim nodes.

Graph Neural Network. To capture causal relations within graphs, GNN is commonly adopted. Liu et al. [134] propose an automated attack detection and investigation method via learning the context semantics of the provenance graph. The provenance graph analyzed by struc2vec captures temporal and causal dependencies of system events. Kairos [29] is a practical intrusion detection and investigation tool based on whole-system provenance. Kairos utilizes GNN to analyze system execution history, so that detects and reconstructs complex APTs. It employs a GNN-based encoder-decoder architecture to learn the temporal evolution of provenance graph structure changes and quantify the abnormal degree of each system event. TREC [139] abstracts APT attack investigation problem as a tactics / techniques recognition problem. TREC trains its model in a few-shot learning manner by adopting a Siamese neural network. Orthurus [100] identifies Quality of Attribution as the key factor contributing to whether or not the industry adopts IDS. It first detects malicious hosts using a GNN encoder and then reconstructs the attack path through dependency analysis. Slot [176], based on provenance graphs and graph reinforcement learning, uncovers hidden relationships among system behaviors, dynamically adapts to new activities and attack strategies, resists adversarial attacks, and automatically constructs attack chains. FeCoGraph [146] directly processes traffic embedding through line graphs to adapt to various GNNs, covering more attack scenarios while protecting data privacy.

6 BENCHMARK DATASETS

DL-IDS relies on high-quality data to train an effective model. This section introduces the dimensions of datasets (Section 6.1) and some public datasets widely used in DL-IDS (Section 6.2).

6.1 Dimensions of Datasets

To illustrate the quality of DL-IDS datasets, it is general to use the following dimensions:

- **Benign Scenarios:** Benign data should cover benign behaviors and system activities to the greatest extent, enabling DL-IDS to learn patterns of benign behaviors to differentiate malicious behaviors.
- **Malicious Scenarios:** Malicious data ought to incorporate typical attack scenarios while taking into account the diversity of attacks, including short-term and long-term attacks, as well as simple attacks and multi-stage attacks.
- **Ground-truth Labels:** Data should be labeled as benign or malicious. For multi-stage attacks, it is useful to indicate the attack type or the attack stage it belongs to.
- **Data Granularities:** Datasets can be in the form of different granularities. The most accepted one is to provide raw log data. Due to copyright concerns, some replicates [41, 99] merely provide post-processed log data without their processing source codes.
- **Operating Systems:** The operating system determines the generalizability of the dataset. The more operating systems a dataset covers and the more common they are, the more comprehensively it can evaluate PIDS performance.

6.2 Public Datasets

Publicly available datasets bring a lot of convenience to research on DL-IDS. However, some researchers use self-made datasets that are not publicly available, making it difficult for other researchers to reuse their datasets [46]. To address this issue, we collect and organize some open-source datasets for further studies, which are listed in Table 6.

LANL Dataset [103] is collected within the internal computer network of Los Alamos National Laboratory's corporate. The dataset consists of 58 consecutive days of de-identified data, covering about 165 million events from 12 thousand users. To obtain, its data sources include Windows-based

Table 6. Overview of public datasets. W, L, F, A, M, and S represent the operating system of Windows, Linux, FreeBSD, Android, Mac, and supercomputer, respectively.

Dataset	Release	Size	Scenarios	Label	Format	System
LANL Dataset [103]	2015	12 GB	-	Yes	.txt	W
StreamSpot [145]	2016	2 GB	1	Yes	.tsv	L
AWSCTD [22]	2018	39 GB	-	No	SQLite	W
DARPA TC E3 [38]	2018	366 GB [67]	6	No	CDM	W, L, F, A
DARPA TC E5 [38]	2019	2,699 GB [67]	8	No	CDM	W, L, F, A
DARPA OpTC [37]	2020	1,100 GB [13]	-	No	eCAR	W
Unicorn SC [76]	2020	147 GB	2	Yes	CDM	L
CERT Dataset [63, 131]	2020	87 GB	-	Yes	.csv	W
LogChunks [20]	2020	24.1 MB	-	Yes	.txt	-
Loghub [266]	2020	77 GB	-	-	.txt	W, L, M, S
ATLAS [9]	2021	0.5 GB	10	Yes	.txt	W
ATLASv2 [184]	2023	12	10	Yes	.txt	W
ProvSec [197]	2023	-	11	Yes	.json	L
AutoLabel [173]	2025	136 GB	29	Yes	.json	L

authentication events, process start and stop events, DNS lookups, network flows, and a set of well-defined red teaming events.

StreamSpot dataset [145] is made up of 1 attack and 5 benign scenarios. The attack scenario exploits a Flash vulnerability and gains root access to the visiting host by visiting a malicious drive-by download URL. The benign scenarios are relevant to normal browsing activity, specifically watching YouTube, browsing news pages, checking Gmail, downloading files, and playing a video game. All the scenarios are simulated through 100 automated tasks with the Selenium RC [208].

DARPA TC datasets [38] are sourced from the DARPA Transparent Computing (TC) program, identified by the number of engagements from E1 to E5. Among them, DARPA TC E3 is the most widely used. The TC program aims to make current computing systems transparent by providing high-fidelity visibility during system operations across all layers of software abstraction. Unfortunately, DARPA TC datasets are released without labels, and DARPA makes no warranties as to the correctness, accuracy, or usefulness of the datasets.

DARPA Operationally Transparent Cyber (OpTC) [37] is a technology transition pilot study funded under Boston Fusion Corporate. The OpTC system architecture is based on the one used in TC program evaluation. In OpTC, every Windows 10 endpoint is equipped with an endpoint sensor that monitors post events, packs them into JSON records, and sends them to Kafka. A translation server aggregates the data into eCAR format and pushes them back to Kafka. OpTC scales TC components from 2 to 1,000 hosts. The dataset consists of approximately 1 TB of compressed JSON data in a highly instrumented environment over two weeks.

Unicorn SC [76] is a dataset specifically designed for APT detection, proposed by Han et al., authors of the Unicorn model. The dataset includes two supply chain scenarios, wget and shell shock, where each scenario lasts for 3 days to simulate the long-term feature of APT attacks, resulting in provenance data containing 125 benign behaviors and 25 malicious behaviors. The data is saved in the form of provenance graphs, describing the causal relationships during the system execution process.

CERT Dataset [131] is a collection of synthetic insider threat test datasets that provide both background and malicious actor synthetic data. It is developed by the CERT Division, in collaboration with ExactData, LLC, and under sponsorship from DARPA I2O. CERT dataset learned

important lessons about the benefits and limitations of synthetic data in the cybersecurity domain and carefully discussed models of realism for synthetic data.

LogChunks [20] is an application log dataset for build log analysis, containing 797 annotated Travis CI build logs from 80 GitHub repositories and 29 programming languages. These logs are from mature and popular projects, collected through repository, build, and log sampling. Each log in the dataset has manually labeled text blocks of build failure reasons, search keywords, and structural categories, and cross-validated with the original developers with an accuracy of 94.4%.

Loghub dataset [266] is a large collection of system log datasets, providing 19 real-world log data from various software systems, including distributed systems, supercomputers, operating systems, mobile systems, server applications, and standalone software. The objective of Loghub is to fill the significant gap between intelligent automated log analysis techniques and successful deployments in the industry. For the usage scenarios of Loghub, about 35% are anomaly detection, 13% are log analysis, and 8% are security.

ATLAS dataset [9] implements 10 attacks based on their detailed reports on real-world APT campaigns and generates audit logs in a controlled testbed environment. Among the ten attacks, four are from single host and the rest six are from multiple hosts. All attacks were developed and executed on Windows 7 32-bit virtual machines and took an hour to complete, along with a 24-hour-window audit logs for benign system behaviors.

ATLASv2 dataset [184] enriches the ATLAS dataset with higher quality background noise and additional logging vantage points. In this dataset, two researchers use the victim machines as their primary work stations throughout the course of engagement, instead of depending on automated scripts to generate activity. System logging, in contrast, cover a five-day period, where the first four days simulate normal work days and the fifth day begins with benign activity then transitions into execution of the corresponding attack.

ProvSec dataset [197] is created for system provenance forensic analysis. To fulfill data provenance requirements, ProvSec includes the full details of system calls including system parameters. In ProvSec, 11 realistic attack scenarios with real software vulnerabilities and exploits are used and an algorithm to improve the data quality in the system provenance forensics analysis is presented.

AutoLabel dataset [173] automates fine-grained log labeling by reducing the labeling problem to obtaining an accurate attack subgraph in a provenance graph. Its experiments consist of 29 scenarios, including 25 real CVE vulnerabilities across 12 widely-used applications (spanning 5 programming languages) plus a Sandworm threat simulation by MITRE CTID.

7 CHALLENGES AND FUTURE DIRECTIONS

After the detailed introduction to the data management stage and the intrusion detection stage, as well as the widely-used benchmark datasets, this section further discusses challenges encountered in existing DL-IDS and summarizes the corresponding visions. These include fundamental resources (Section 7.1), pre-trained large models (Section 7.2), and comprehensive applications (Section 7.3).

7.1 Fundamental Resources

Effective DL-IDS heavily depends on core fundamental resources such as datasets and computing facilities to develop [105]. Here, we will discuss their challenges one after the other.

7.1.1 Poor Data Quality. Existing datasets for DL-IDS may contain errors, inaccuracies, or missing values. This leads to unreliable descriptions of system behaviors that may mislead DL-IDS. For example, in some cases of the DARPA TC dataset, the PROCESS object and its source fail to properly resolve conflicts, resulting in possible incorrect transformation. Besides, the `acuity_level` value of the FLOW object is 0, while the value range for this field in other objects is from 1 to 5. Another

example could be the LogChunks [20] dataset. In this dataset, the content describing the failure reasons is possibly incomplete. This is because a chunk in LogChunks only contains a continuous substring of the log text and a failure reason may be described across multiple sections of the log. Moreover, LogChunks neglects the classification of failure reasons like test, compilation, and code inspection errors, which hinders further research from analyzing failure reasons.

Meanwhile, high-quality ground-truth labels are hard to acquire, which is impeded by the contradiction between fine-grained manual labeling and automated label generation. On one hand, for unknown intrusions such as zero-day attacks, it is very labor-intensive for security analysts to correspond each attack scenario to certain log entries, although coarse-grained attack scenarios may have been acquired. The DAPRA TC dataset [38] is a typical example for this. It only provides a ground truth report for attack scenarios, which does not correspond to any specific log entries. Although a few researchers [219] provide the third-party ground-truth labels that are manually identified by themselves, we empirically find some ambiguities between their ground-truth labels and the official attack scenario report. These ambiguities have an obviously negative effect on DL-IDS, and to some extent, they may even cause the accumulation of errors. On the other hand, the development of automated labeling tools is in an awkward position. The log data is generated based on its given prior knowledge of intrusions [28], whereas the challenge of DL-IDS is to detect zero-day intrusions. This tends the development of such automated tools to be somewhat pointless.

In addition, there are no unified and effective evaluation metrics for DL-IDS [29], which further weakens the potential of datasets. For example, precision, recall, F1 score are usually exploited in most studies [9, 99, 182, 216], while some papers [41] propose to use True Positive Rate (TPR) and False Positive Rate (FPR) as evaluation metrics. This makes the comparison experiments usually unfair and hard to tell if the validation is convincing. We also note that in many cases where the percentage of negatives (or malicious log entries) is low, sacrificing FPR can always significantly increase TPR. For example, sacrificing 1,000 false positives for one true positive might only increase FPR by 0.05%, but would increase TPR by 5%.

7.1.2 Insufficient Amount of Data. Although log data is generated very quickly (e.g., eBay generates 1.2 PB log data per day by 2018 [189]), DL-IDS is still facing challenges in insufficient amounts of data. Discounting the above data quality issues such as inaccuracies, the reasons are three-fold:

First, log data has an extremely large number of trivial events, which are proven ineffective and usually removed by graph summarization [237, 250]. For example, data provenance provides fine-grained information about memory-related events, such as data-to-memory mapping and protection of certain memory addresses. These memory-related events basically do not involve attacks, and unfortunately, are always orthogonal to the existing DL-IDS. However, to ensure the completeness requirement of data provenance and to capture very infrequent but inevitable memory attacks, these memory-related events are still recorded in benchmark datasets. As a result, the usable part of each dataset is rather small for DL-IDS, which can be reflected by the high summarization ratio achieved by graph summarization approaches (e.g., 70% [234]).

The second reason for an insufficient amount of data is the limited dataset representativeness. As observed in Table 6, most of the datasets have no more than 10 attack scenarios, not to mention that each of these attack scenarios has been carefully chosen by their authors. This limited number of attack scenarios suggests that existing datasets are almost impossible to represent the diversified attack methods, as the number of CVE records has already been over 280,000 [31]. Furthermore, the existing datasets such as DAPRA TC E3 [38] are collected in a specific experimental environment and may not cover other types of normal system behaviors, and are proven that a significant amount of synthetic data exists [133]. DARPA TC E5 [38] is unusable for most experiments due to the sparse and error-filled documentation. Unicorn SC [76] is generated by an idealized simulation

of supply chain scenarios, which means many real-world features are prone to be ignored in this dataset. Hence, training DL-IDS on these non-representative datasets could be a disaster for the computer systems that they protect.

Finally, the accessibility of datasets further exacerbates the insufficient data problem. Due to privacy and copyright issues, some datasets may be proprietary or difficult to obtain [216, 218]. Moreover, ProvDetector [216] conducted a three-month system evaluation in an enterprise environment with 306 hosts and collected benign provenance data of 23 target programs. Yet this dataset has not been made public, rendering it unavailable to improve other DL-IDS and almost all the assessment settings related to ProvDetector are susceptible to inequity.

7.1.3 Potential Heavy Computation Requirements. Similar to other DL techniques, DL-IDS also requires a potentially large amount of computing resources to improve their performance. According to [185], the generalizability of neural models is proportional to the investment of computing resources. Supposing that the challenge of insufficient data is mitigated and a large volume of log data is available, more computing resources are inevitably required. Besides, we will illustrate in Section 7.2 that there are plenty of powerful techniques that have not been introduced in DL-IDS, which will also bring in computation requirements. Unfortunately, acceleration methods like parallel computation and efficient retrieval have not been fully scheduled by the cybersecurity community. An example is that the computation time of Unicorn equipped with one core is proven linear to its workloads [76]. It is clear that the efficiency of Unicorn, which is not implemented in parallel, will reach the bottleneck as this core does.

7.1.4 Future Directions. To conclude, the challenges for DL-IDS in fundamental resources consist of data quality, data volume, and computational overhead. Apart from unintentional errors and non-technical issues in fundamental resources, the research questions that urgently need to be addressed include the contradiction between unaffordable manual labeling and non-generalizable auto-labeling techniques, non-unified benchmark datasets and evaluation metrics, as well as potential heavy computational overheads. Therefore, we summarize the future directions as follows:

Future Directions

- Developing efficient man-machine interactive log labeling mechanisms and organizing open-source data-sharing platforms accordingly to provide large amounts of high-quality datasets.
- Maintaining effective and comprehensive benchmark datasets, accompanied by a unified performance metric framework for a fair comparison.
- Investigating parallel or simplified strategies for DL-IDS, and studying their integration with log storage systems to achieve end-to-end acceleration.

7.2 Pre-training Theories and Techniques

In recent years, significant progress has been made by Large Language Models (LLMs) in the field of DL. Their capacity to understand and generate dialogue has been greatly enhanced as the model parameters of LLMs keep rising. T5 [179], BERT [39], GPT [178], GPT-4 [2], LaMDA [207], and LLaMA [209] are notable examples.

With the development of pre-training techniques, LLMs have been adopted in many fields such as finance [258], education [164], medicine [172], and even other domains of cybersecurity [34, 69, 92]. In contrast, the adoption of LLMs in DL-IDS is stagnant, as shown in Figure 6. We can observe that LLMs developed at full speed beginning in 2019. Their prosperity, however, has not extended to DL-IDS. Until now, the only two DL-IDS that incorporate pre-training techniques, AirTag [41] and

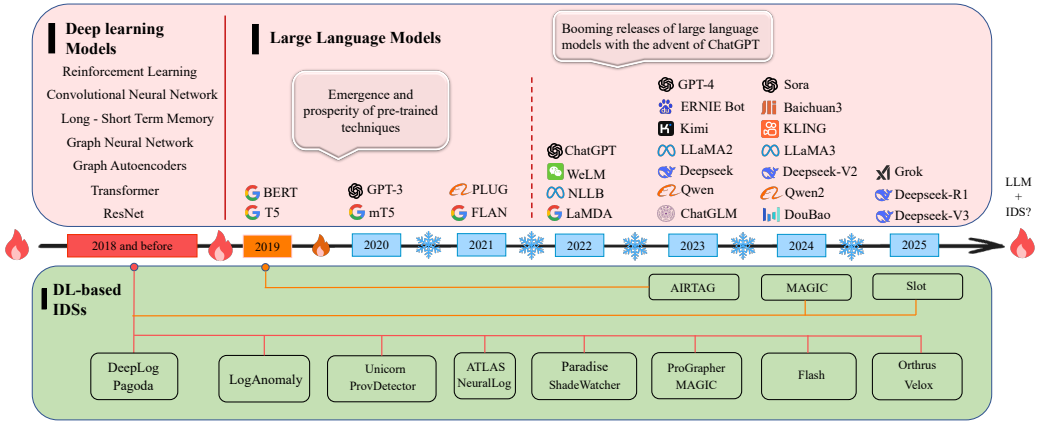


Fig. 6. Interactions between DL models and DL-IDS. While DL models proposed before 2019 have already leveraged in DL-IDS, the emerging LLMs (or pre-training theories and techniques) since 2020 remains underdeveloped in this domain.

MAGIC [99], still do not make full use of the potential of LLMs. AirTag pre-trains a BERT model on application logs and detects intrusions in terms of embeddings generated by BERT. MAGIC introduces GraphMAE [90], a model architecture derived from Graph Autoencoder [109] in 2016 but integrated with the famous masked self-supervised learning method [81] in 2022, to conduct self-supervised learning on provenance graphs. MAGIC further designs an adapter to apply the pre-trained model in different detection scenarios. Nevertheless, both AirTag and MAGIC can be regarded as preliminary explorations of pre-training techniques. According to the scaling law [102], the performance of LLMs will steadily improve, as the parameters, data, and computation increase. And the reasoning ability of LLMs will suddenly emerge [220], allowing them to chat with humans smoothly. Such advantageous abilities obviously have not been incorporated into DL-IDS.

Nowadays, some researchers [7, 59, 125, 160] have started to explore the applications of LLMs on DL-IDS. Yet the theories and techniques of such combination remain challenges. In the following, we will illustrate the identified issues and then point out the future directions.

7.2.1 Trade-off between Reliability and Generalizability. The governing concern for the employment of LLMs in DL-IDS is reliability (or explainability). Although offering generalizability, LLMs have long been denounced to have issues with hallucinations [149, 241], privacy [84, 240, 244], overreliance [107], and backdoor threats [136]. These unexplainable and uncontrollable features are an absolute disaster for DL-IDS. For example, when feeding log data to LLMs, they sometimes are prone to hallucinate and provide wrong detection results. Attacks thus successfully bypass the detection facilities and can exfiltrate sensitive data in the victim computer systems. Another example for this is that sensitive information may leak from LLMs. Hui et al. [93] present a prompt leakage attack for LLMs, which is demonstrated to be effective in both offline settings and real-world LLM applications.

7.2.2 Short of Statistical Log Modeling. LLMs are developed on the basis of statistical language modeling [101, 187], which is not insufficiently studied for log data. The statistical modeling of natural language can be traced back to the early 1950s when Shannon pioneered the technique of predicting the next element of natural language text [195] and discussed the n-gram model for

Table 7. Comparison of research advances in statistical modeling of various data. “NL”, “PL” and “FL” represent Natural Language, Programming Language, and Formal Language, respectively. Note that PL is a type of FL.

Data	Form	Content Generation Rules	Statistical Modeling Studies	Pre-training
Text	NL	Grammar, pragmatics, semantics, etc	[101, 148, 187, 196]	well-done
Speech	NL	Text rules (see above) and phonetics	[104, 167]	well-done
Source code	PL	Lexical and syntactic definitions	[8, 85, 180]	well-done
Log	NL + FL	Log template defined by developers	future work	underdeveloped

English [196]. After that, as machine learning came into view of the NLP research communities, language modeling flourished, and many models such as TreeBank [148], word2vec [154, 155] and LSTM [86] were proposed. Over decades, researchers in NLP have gained solid knowledge of language modeling, whose interests gradually shifted to efficiency. An epoch-making model, Transformer [212], was presented using the multi-head self-attention mechanism to fulfill parallel computing, which was widely exploited in popular pre-trained models such as BERT [39] and GPT [2] afterward. It is evident that the success of LLMs comes from the prolonged studies on statistical language modeling.

Unfortunately, there are almost no research efforts on statistical modeling of log data, resulting in pre-training techniques of DL-IDS remaining underdeveloped. By contrast, as illustrated in Table 7, the statistical modeling studies of other types of data have already started. Hindle et al. [85] demonstrate that the source code is very repetitive and predictable, and, in fact, even more so than natural language. Driven by such statistical modeling conclusion, DL-based source code applications [54, 70, 124, 126, 203, 233, 235] such as code generation and code clone detection flourish, many of which have already becomes common applications in LLMs. Similar cases can be found for speech data, whose applications are like text to speech [71, 169, 183] and speech recognition [14].

We argue that log data is also created by humans, similar to text, speech, and source code. It is generated according to developer-defined log templates, with a form of both natural language (e.g., application logs) and formal language (e.g., data provenance in CDM format). Given the fact that natural language (e.g., text and speech) and formal language (e.g., source code) both exhibit positive performance in pre-training, log data urgently demands statistical modeling achievements to facilitate its pre-training research. Although several works [96, 152] have discussed the features of log data, they are orthogonal to the explainable combination of DL and IDS. Compared with the other data types, challenges in statistical log modeling, for instance, may lie in that logs are extremely long and detailed for reliable purposes. It is very common that the length of one single log entry is the same as that of one paragraph in natural language texts. These challenges happen to be the shortcomings of LLMs - not being able to handle long text and not being trustworthy in generated contents.

7.2.3 Future Directions. According to the scaling laws [102] and emergent abilities theory [220], as the model size continues to grow, the performance of DL-IDS will increase simultaneously. Thus, increasing the amount of model parameters will be an inevitable trend for DL-IDS. The underlying research questions include the strategies for incorporating existing LLMs in intrusion detection, since it is infeasible to directly leverage unreliable LLMs to detect intrusions, and the theories and techniques for modeling long and detailed log data. We summarize the future directions as follows:

Future Directions

- Investigating how and where to introduce LLMs into DL-IDS like [165], with the objective of balancing the generalizability provided by LLMs and the reliability required by DL-IDS.
- Exploring fundamental statistical modeling theories for log data. On this basis, designing pre-training frameworks for log data and its downstream tasks such as steps within the workflow of DL-IDS (see Section 3.2).

7.3 Comprehensive Applications and Scenarios

DL-IDS possess abilities that the traditional IDS lack, or are difficult to realize, such as generalizability for zero-day attacks and modeling ability for complicated downstream tasks. We will elaborate on the possible new-style applications and discuss the challenges in and introduced by them.

7.3.1 Limited Forward and Backward Tracing Scope. Forward tracing and backward tracing are employed in attack investigation, as illustrated in Section 5.3. Under traditional settings, the forward tracing analyzes the influence a symptom node would have on the victim computer system, and the backward tracing discovers the starting node where the vulnerabilities exist [270].

We argue that the existing tracing scope is too limited to handle increasingly complicated intrusions and DL-IDS can be defined more broadly. In addition to investigating scenario graphs of intrusions, DL-IDS are supposed to further investigate why these intrusions occur and how to hold back them. The broader definition introduces more downstream tasks that would be difficult to accomplish without the assistance of DL techniques. Based on Definition 3.3, we reformulate the definition of intrusion in a broad sense for DL-IDS as follows:

Definition 7.1. (Generalized Intrusion). Generalized intrusion is the malicious attempts against a computer, a network, or the corresponding security facilities, whose attributes encompass not only itself but also its underlying root causes and the relevant control measures.

In this way, the detection of DL-IDS has been extended to the broadly defined intrusions, including their attributes of both root causes and control measures. When executing backward tracing analysis, DL-IDS are not only required to detect the starting symptom nodes of intrusions, but also required to find the root causes of these symptom nodes (i.e., vulnerabilities in source codes). In the forward tracing analysis, except for detecting the symptom nodes affected by intrusions, DL-IDS should perform an in-depth analysis to discover the potentially compromised nodes and provide control measures for handling intrusions.

Thankfully, several pioneering works have studied similar problems [25, 144]. In AiVl [25], algorithms to bridge log entries and program models are developed using dynamic-static program analysis. Root causes for the exploited vulnerabilities are capable of directly deriving from intrusion detection. Pedro et al. [144] investigate detection and mitigation methods for DDoS attacks, aiming to control them immediately. Additionally, semi-automated adaptive network defense (SAND) [26] leverages SDN to dynamically generate and deploy defense rules. We note that these research attempts are all based on heuristics, either using pre-defined rules to generate root causes, or developing control measures for specific intrusions. Thus, there is a substantial need to introduce advanced DL techniques to this problem.

7.3.2 Concerns about Data-driven Adversarial Attacks. To validate the detection performance, DL-IDS commonly idealize the experimental data in their threat model. Such idealization, however, leaves DL-IDS with weaknesses that could be exploited by invaders. For example, a common assumption is that no attacks are considered to compromise the security of the log collection

systems [76, 79, 99, 182], namely log data utilized in DL-IDS is absolutely harmless. But as attacks become more stealthy and complicated, it is impossible to satisfy such an assumption apparently. When DL-IDS encounter intentional data poisoning attacks, prediction backdoors could be easily planted as persistent vulnerabilities.

The robustness of DL-IDS is also challenged by data-driven evasion attacks. To evade the detection, the malicious behaviors usually mimic the benign ones (a.k.a., mimicry attacks), making them hard to be detected. By early 2002, David et al. [215] have indicated the danger of mimicry attacks on HIDS. Recently, researchers have started to investigate mimicry attacks on DL-IDS [64, 132, 161] and their studies all present effective evasion of detection. From a study [24], DL-IDS can be even plagued by a trivial perturbation in log data. Aware of this issue, R-caid [65] proposes to embed root causes into the detection model for countering adversarial attacks. However, as noted in recent work [64, 65, 161], data-driven attacks still remain a major challenge for DL-IDS.

7.3.3 Underexplored Promising Scenarios. While DL-IDS show excellent performance in the protection of computer and network systems recently, there are still many promising scenarios for DL-IDS that have not been explored sufficiently.

Mobile edge computing (MEC) [1, 117, 147] is a typical scenario. In the MEC environment, mobile computing, network control, and storage are pushed at the network edges so as to enable computation-intensive tasks at the resource-limited devices. At the network edges, devices such as Unmanned Aerial Vehicles (UAVs) and New Energy Vehicles (NEVs) usually lack computing power and security facilities, making it difficult to prevent them from intrusions [198]. In the meantime, containerized deployment has become one of the dominant ways to deploy microservices. Detecting intrusions on containers is thus of great importance, for which ReplicaWatcher [46] is a representative work with a special design for microservices. Additionally, industrial networks are characterized by high fidelity, stability, and real-time responsiveness [110], leading to challenges in adapting DL-IDS to their infrastructures.

7.3.4 Future Directions. Although there has been plenty of research on DL-IDS, many applications and scenarios remain underdeveloped. DL-IDS are sought to be more broadly defined and applied. Based on the above discussion, we briefly summarize the future directions as follows:

Future Directions

- Extending the scope of forward tracing and backward tracing to intrusions in a broad sense, so that generating root causes and control measures for the broadly defined intrusions.
- Understanding data-driven adversarial attacks such as data poisoning attacks and mimicry attacks for devising more robust DL-IDS.
- Applying DL-IDS widely in more underexplored promising scenarios, and if possible, implementing unified frameworks for them.

8 CONCLUSION

The DL techniques bring reform to IDS, whose generalizability enables them to detect intrusions that have never been encountered before. Recognizing that the IDS development over the past decade primarily comes from DL-IDS, this survey revisits the common workflow for DL-IDS, elaborates each module in the workflow, and taxonomizes the research papers innovatively based on their DL techniques. Publicly available datasets for stimulating future research are introduced subsequently. In addition, from the perspective of DL, this survey digs deep into the potential challenges, emerging trends, and future directions for DL-IDS. The discussions suggest to us that

DL-IDS are, fascinatingly, in an underdeveloped state. We hope that this survey can somewhat inspire current researchers and facilitate future investigations on DL-IDS.

ACKNOWLEDGMENTS

This research is sponsored in part by the NSFC program (No. 6212780016 and No. 62021002).

REFERENCES

- [1] Nasir Abbas, Yan Zhang, Amir Taherkordi, and Tor Skeie. 2017. Mobile Edge Computing: A Survey. *IEEE Internet of Things Journal* 5, 1 (2017), 450–465.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Amey Agrawal, Rohit Karlupia, and Rajat Gupta. 2019. Logan: A Distributed Online Log Parser. In *Proceedings of the 2019 IEEE 35th International Conference on Data Engineering*. IEEE, 1946–1951.
- [4] Zeeshan Ahmad, Adnan Shahid Khan, Cheah Wai Shiang, Johari Abdullah, and Farhan Ahmad. 2021. Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches. *Transactions on Emerging Telecommunications Technologies* 32, 1 (2021), e4150.
- [5] Farrukh Ahmed, Urooj Jahangir, Hamad Rahim, Kamran Ali, et al. 2020. Centralized Log Management Using Elasticsearch, Logstash and Kibana. In *Proceedings of the 2020 International Conference on Information Science and Communication Technology*. IEEE, 1–7.
- [6] Mohannad Alhanahnah, Shiqing Ma, Ashish Gehani, Gabriela F Ciocarlie, Vinod Yegneswaran, Somesh Jha, and Xiangyu Zhang. 2022. autoMPI: Automated Multiple Perspective Attack Investigation with Semantics Aware Execution Partitioning. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2761–2775.
- [7] Tarek Ali. 2024. *Next-Generation Intrusion Detection Systems with LLMs: Real-Time Anomaly Detection, Explainable AI, and Adaptive Data Generation*. Master's thesis. T. Ali.
- [8] Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. 2018. A Survey of Machine Learning for Big Code and Naturalness. *ACM Computing Surveys* 51, 4 (2018), 1–37.
- [9] Abdullellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. ATLAS: A Sequence-based Learning Approach for Attack Investigation. In *Proceedings of the 30th USENIX Security Symposium*. 3005–3022.
- [10] Adel Alshamrani, Sowmya Myneni, Ankur Chowdhary, and Dijiang Huang. 2019. A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities. *IEEE Communications Surveys and Tutorials* 21, 2 (2019), 1851–1877. <https://doi.org/10.1109/COMST.2019.2891891>
- [11] Enes Altinisik, Fatih Deniz, and Hüseyin Taha Sencar. 2023. ProvG-Searcher: A Graph Representation Learning Approach for Efficient Provenance Graph Search. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2247–2261.
- [12] Clarivate Analytics. 1997. *Web of Science*. <https://www.webofscience.com>
- [13] Md Monowar Anjum, Shahrear Iqbal, and Benoit Hamelin. 2021. Analyzing the Usefulness of the DARPA OpTC Dataset in Cyber Threat Detection Research. In *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies*. 27–32.
- [14] Alexei Baevski, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. 2021. Unsupervised Speech Recognition. *Advances in Neural Information Processing Systems* 34 (2021), 27826–27839.
- [15] Elizabeth Bautista, Nitin Sukhija, and Siqi Deng. 2022. Shasta Log Aggregation, Monitoring and Alerting in HPC Environments with Grafana Loki and ServiceNow. In *Proceedings of the 2022 IEEE International Conference on Cluster Computing*. IEEE, 602–610.
- [16] Jack Beerman, David Berent, Zach Falter, and Suman Bhunia. 2023. A Review of Colonial Pipeline Ransomware Attack. In *Proceedings of the 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops*. IEEE, 8–15.
- [17] Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, and Anis Zouaoui. 2023. Graph Neural Networks for Intrusion Detection: A Survey. *IEEE Access* 11 (2023), 49114–49139.
- [18] Tristan Bilot, Baoxiang Jiang, Zefeng Li, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, and Thomas Pasquier. 2025. Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based Intrusion Detection Systems. In *34th USENIX Security Symposium (USENIX Security 25)*. 7193–7212.
- [19] Peter Bodik, Moises Goldszmidt, Armando Fox, Dawn B Woodard, and Hans Andersen. 2010. Fingerprinting the Datacenter: Automated Classification of Performance Crises. In *Proceedings of the 5th European Conference on Computer Systems*. 111–124.

- [20] Carolin E Brandt, Annibale Panichella, Andy Zaidman, and Moritz Beller. 2020. LogChunks: A Data Set for Build Log Analysis. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 583–587.
- [21] Robert A Bridges, Tarrah R Glass-Vanderlan, Michael D Iannaccone, Maria S Vincent, and Qian Chen. 2019. A Survey of Intrusion Detection Systems Leveraging Host Data. *ACM computing surveys* 52, 6 (2019), 1–35.
- [22] Dainius Čeponis and Nikolaj Goranin. 2018. Towards A Robust Method of Dataset Generation of Malicious Activity for Anomaly-Based HIDS Training and Presentation of AWSCTD Dataset. *Baltic Journal of Modern Computing* 6, 3 (2018), 217–234.
- [23] Xiaolin Chai, Hang Zhang, Jue Zhang, Yan Sun, and Sajal K Das. 2024. Log Sequence Anomaly Detection based on Template and Parameter Parsing via BERT. *IEEE Transactions on Dependable and Secure Computing* (2024).
- [24] Anirban Chakraborty, Manaar Alam, Vishal Dey, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. 2018. Adversarial Attacks and Defences: A Survey. *arXiv preprint arXiv:1810.00069* (2018).
- [25] Changhua Chen, Tingzhen Yan, Chenxuan Shi, Hao Xi, Zhirui Fan, Hai Wan, and Xibin Zhao. 2024. The Last Mile of Attack Investigation: Audit Log Analysis towards Software Vulnerability Location. *IEEE Transactions on Information Forensics and Security* (2024).
- [26] Haoyu Chen, Deqing Zou, Hai Jin, Shouhuai Xu, and Bin Yuan. 2022. SAND: Semi-Automated Adaptive Network Defense via Programmable Rule Generation and Deployment. *Science China Information Sciences* 65, 7 (2022), 172102.
- [27] Tao Chen, Haiyan Suo, and Wenqian Xu. 2023. Design of Log Collection Architecture Based on Cloud Native Technology. In *Proceedings of the 2023 4th Information Communication Technologies Conference*. IEEE, 311–315.
- [28] Wenrui Cheng, Qixuan Yuan, Tiantian Zhu, Tieming Chen, Jie Ying, Aohan Zheng, Mingjun Ma, Chunlin Xiong, Mingqi Lv, and Yan Chen. 2025. TAGAPT: Towards Automatic Generation of APT Samples with Provenance-level Granularity. *IEEE Transactions on Information Forensics and Security* (2025).
- [29] Zijun Cheng, Qiujuan Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. Kairos: Practical Intrusion Detection and Investigation Using Whole-System Provenance. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy*. IEEE, 3533–3551.
- [30] Guojun Chu, Jingyu Wang, Qi Qi, Haifeng Sun, Shimin Tao, and Jianxin Liao. 2021. Prefix-Graph: A Versatile Log Parsing Approach Merging Prefix Tree with Probabilistic Graph. In *Proceedings of the 2021 IEEE 37th International Conference on Data Engineering*. IEEE, 2411–2422.
- [31] The MITRE Corporation. 2025. *CVE List*. <https://github.com/CVEProject/cvelistV5/archive/refs/heads/main.zip>
- [32] Oihana Coustié, Josiane Mothe, Olivier Teste, and Xavier Baril. 2020. METING: A Robust Log Parser Based on Frequent n-Gram Mining. In *Proceedings of the 2020 IEEE International Conference on Web Services*. IEEE, 84–88.
- [33] Jian Cui, Hanna Kim, Eugene Jang, Dayeon Yim, Kicheol Kim, Yongjae Lee, Jin-Woo Chung, Seungwon Shin, and Xiaojing Liao. 2024. Tweezers: A Framework for Security Event Detection via Event Attribution-centric Tweet Embedding. In *Proceedings of the Network and Distributed System Security Symposium*.
- [34] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2025. LLM Compiler: Foundation Language Models for Compiler Optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction*. 141–153.
- [35] Hetong Dai, Heng Li, Che-Shao Chen, Weiyl Shang, and Tse-Hsun Chen. 2020. Logram: Efficient Log Parsing Using n -Gram Dictionaries. *IEEE Transactions on Software Engineering* 48, 3 (2020), 879–892.
- [36] Hetong Dai, Yiming Tang, Heng Li, and Weiyl Shang. 2023. PILAR: Studying and Mitigating the Influence of Configurations on Log Parsing. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 818–829.
- [37] DARPA. 2019. *Operationally Transparent Cyber Dataset*. <https://github.com/FiveDirections/OpTC-data>
- [38] DARPA. 2022. *The DARPA Transparent Computing (TC) program Data Release*. <https://github.com/darpa-i2o/Transparent-Computing>
- [39] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.
- [40] Hailun Ding, Juan Zhai, Dong Deng, and Shiqing Ma. 2023. The Case for Learned Provenance Graph Storage Systems. In *Proceedings of the 32nd USENIX Security Symposium*. 3277–3294.
- [41] Hailun Ding, Juan Zhai, Yuhong Nan, and Shiqing Ma. 2023. AirTag: Towards Automated Attack Investigation by Unsupervised Learning with Log Texts. In *Proceedings of the 32nd USENIX Security Symposium*. 373–390.
- [42] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. DistDet: A Cost-Effective Distributed Cyber Threat Detection System. In *Proceedings of the 32nd USENIX Security Symposium*. 6575–6592.
- [43] Ying Dong, Yuqing Zhang, Hua Ma, Qianru Wu, Qixu Liu, Kai Wang, and Wenjie Wang. 2018. An Adaptive System for Detecting Malicious Queries in Web Attacks. *Science China Information Sciences* 61, 3 (2018), 032114.

- [44] Min Du and Feifei Li. 2016. Spell: Streaming Parsing of System Event Logs. In *Proceedings of the 2016 IEEE 16th International Conference on Data Mining*. IEEE, 859–864.
- [45] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1285–1298.
- [46] Asbat El Khairi, Marco Caselli, Andreas Peter, and Andrea Continella. 2024. REPLICAWATCHER: Training-less Anomaly Detection in Containerized Microservices. In *Proceedings of the Network and Distributed System Security Symposium*.
- [47] Elastic. 2009. *Logstash: Collect, parse, and transform logs*. <https://www.elastic.co/logstash/>
- [48] Elastic. 2010. *Elasticsearch: The official distributed search & analytics engine*. <https://www.elastic.co/elasticsearch/>
- [49] Elastic. 2013. *Kibana: Explore, visualize, and discover data*. <https://www.elastic.co/kibana/>
- [50] Elsevier. 2021. *Scopus*. [#basic](https://www.scopus.com/search/form.uri?display=basic)
- [51] Dave Evans. 2012. The Internet of Everything: How More Relevant and Valuable Connections will Change the World. *Cisco IBSG 2012* (2012), 1–9.
- [52] Pengcheng Fang, Peng Gao, Changlin Liu, Erman Ayday, Kangkook Jee, Ting Wang, Yanfang Fanny Ye, Zhuotao Liu, and Xusheng Xiao. 2022. Back-Propagating System Dependency Impact for Attack Investigation. In *Proceedings of the 31st USENIX Security Symposium*. 2461–2478.
- [53] Peng Fei, Zhou Li, Zhiying Wang, Xiao Yu, Ding Li, and Kangkook Jee. 2021. SEAL: Storage-Efficient Causality Analysis on Enterprise Logs with Query-Friendly Compression. In *Proceedings of the 30th USENIX Security Symposium*.
- [54] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547.
- [55] Free Software Foundation. 1992. *gzip: GNU zip compression utility*. <https://www.gnu.org/software/gzip/>
- [56] Chuanpu Fu, Qi Li, Meng Shen, and Ke Xu. 2021. Realtime Robust Malicious Traffic Detection via Frequency Domain Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3431–3446.
- [57] Chuanpu Fu, Qi Li, Meng Shen, and Ke Xu. 2024. Detecting Tunneled Flooding Traffic via Deep Semantic Analysis of Packet Length Patterns. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 3659–3673.
- [58] Chuanpu Fu, Qi Li, Ke Xu, and Jianping Wu. 2023. Point Cloud Analysis for ML-based Malicious Traffic Detection: Reducing Majorities of False Positive Alarms. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1005–1019.
- [59] Oscar G. Lira, Alberto Marroquin, and Marco Antonio To. 2024. Harnessing the Advanced Capabilities of LLM for Adaptive Intrusion Detection Systems. In *Proceedings of the International Conference on Advanced Information Networking and Applications*. Springer, 453–464.
- [60] Peng Gao, Xusheng Xiao, Zhichun Li, Fengyuan Xu, Sanjeev R Kulkarni, and Prateek Mittal. 2018. AIQL: Enabling Efficient Attack Investigation from System Monitoring Data. In *Proceedings of the 2018 USENIX Annual Technical Conference*. 113–126.
- [61] Ashish Gehani and Dawood Tariq. 2012. SPADE: Support for Provenance Auditing in Distributed Environments. In *Proceedings of the ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing*. Springer, 101–120.
- [62] Jalal Ghadermazi, Soumyadeep Hore, Ankit Shah, and Nathaniel D Bastian. 2025. GTAE-IDS: Graph Transformer-Based Autoencoder Framework for Real-Time Network Intrusion Detection. *IEEE Transactions on Information Forensics and Security* (2025).
- [63] Joshua Glasser and Brian Lindauer. 2013. Bridging the gap: A Pragmatic Approach to Generating Insider Threat Data. In *Proceedings of the IEEE Symposium on Security and Privacy Workshops*. IEEE, 98–104.
- [64] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. 2023. Sometimes, You Aren’t What You Do: Mimicry Attacks Against Provenance Graph Host Intrusion Detection Systems. In *Proceedings of the Network and Distributed System Security Symposium*.
- [65] Akul Goyal, Gang Wang, and Adam Bates. 2024. R-caid: Embedding Root Cause Analysis within Provenance-Based Intrusion Detection. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy*. IEEE, 3515–3532.
- [66] Brendan Gregg and Jim Mauro. 2011. *DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X, and FreeBSD*. Prentice Hall Professional.
- [67] John Griffith, Derrick Kong, Armando Caro, Brett Benyo, Joud Khoury, Timothy Upthegrove, Timothy Christovich, Stanislav Ponomorov, Ali Sydney, Arjun Saini, et al. 2020. Scalable Transparency Architecture for Research Collaboration (STARC)-DARPA Transparent Computing (TC) Program. *Raytheon BBN Technologies Corporation Cambridge United States* (2020).
- [68] Steve Grubb. 2008. *Linux audit*. <https://people.redhat.com/sgrubb/audit/>

- [69] Qiuhan Gu. 2023. LLM-Based Code Generation Method for Golang Compiler Testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2201–2203.
- [70] Xiaodong Gu, Meng Chen, Yalan Lin, Yuhan Hu, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, and Juhong Wang. 2025. On the Effectiveness of Large Language Models in Domain-Specific Code Generation. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–22.
- [71] Yiwei Guo, Chenpeng Du, Ziyang Ma, Xie Chen, and Kai Yu. 2024. Voiceflow: Efficient Text-to-Speech with Rectified Flow Matching. In *Proceedings of the ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 11121–11125.
- [72] Yi Guo, Fu Miao, Liancheng Zhang, and Yu Wang. 2019. CATH: An Effective Method for Detecting Denial-of-Service Attacks in Software Defined Networks. *Science China Information Sciences* 62, 3 (2019), 32106.
- [73] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems* 30 (2017).
- [74] Hossein Hamooni, Biplob Debnath, Jianwu Xu, Hui Zhang, Guofei Jiang, and Abdullah Mueen. 2016. LogMine: Fast Pattern Recognition for Log Analytics. In *Proceedings of the ACM International Conference on Information and Knowledge Management*. 1573–1582.
- [75] Dongqi Han, Zhiliang Wang, Wenqi Chen, Kai Wang, Rui Yu, Su Wang, Han Zhang, Zhihua Wang, Minghui Jin, Jiahai Yang, et al. 2023. Anomaly Detection in the Open World: Normality Shift Detection, Explanation, and Adaptation. In *Proceedings of the Network and Distributed Systems Security Symposium*.
- [76] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. 2020. Unicorn: Runtime Provenance-Based Detector for Advanced Persistent Threats. In *Proceedings of the Network and Distributed Systems Security Symposium*.
- [77] Wajih Ul Hassan, Lemay Aguse, Nuraini Aguse, Adam Bates, and Thomas Moyer. 2018. Towards Scalable Cluster Auditing through Grammatical Inference over Provenance Graphs. In *Proceedings of the Network and Distributed Systems Security Symposium*.
- [78] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical Provenance Analysis for Endpoint Detection and Response Systems. In *Proceedings of the 2020 IEEE symposium on security and privacy*. IEEE, 1172–1189.
- [79] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. Nodose: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *Proceedings of the Network and Distributed System Security Symposium*.
- [80] Wajih Ul Hassan, Mohammad Ali Nouredine, Pubali Datta, and Adam Bates. 2020. OmegaLog: High-Fidelity Attack Investigation via Transparent Multi-Layer Log Analysis. In *Proceedings of the Network and Distributed System Security Symposium*.
- [81] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. 2022. Masked Autoencoders are Scalable Vision Learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16000–16009.
- [82] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An Online Log Parsing Approach with Fixed Depth Tree. In *Proceedings of the 2017 IEEE International Conference on Web Services*. IEEE, 33–40.
- [83] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. 2020. A Survey on Automated Log Analysis for Reliability Engineering. *ACM Computing Surveys* 54 (2020), 1 – 37. <https://api.semanticscholar.org/CorpusID:221703032>
- [84] Xinlei He, Guowen Xu, Xingshuo Han, Qian Wang, Lingchen Zhao, Chao Shen, Chenhao Lin, Zhengyu Zhao, Qian Li, Le Yang, et al. 2025. Artificial intelligence security and privacy: a survey. *Science China Information Sciences* 68, 8 (2025), 1–90.
- [85] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the Naturalness of Software. *Commun. ACM* 59, 5 (2016), 122–131.
- [86] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780.
- [87] Josef Horalek, Patrik Urbanik, Vladimir Sobeslav, and Tomas Svoboda. 2022. Proposed Solution for Log Collection and Analysis in Kubernetes Environment. In *Proceedings of the International Conference on Nature of Computation and Communication*. Springer, 9–22.
- [88] Md Nahid Hossain, Sadegh M Milajerdi, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R Sekar, Scott Stoller, and VN Venkatakrishnan. 2017. Sleuth: Real-time Attack Scenario Reconstruction from COTS Audit Data. In *Proceedings of the USENIX Security Symposium*. 487–504.
- [89] Md Nahid Hossain, Junao Wang, Ofir Weisse, R Sekar, Daniel Genkin, Boyuan He, Scott D Stoller, Gan Fang, Frank Piessens, Evan Downing, et al. 2018. Dependence-Preserving Data Compaction for Scalable Forensic Analysis. In *Proceedings of the 27th USENIX Security Symposium*. 1723–1740.

- [90] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. GraphMAE: Self-Supervised Masked Graph Autoencoders. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 594–604.
- [91] Kevin Hsieh, Mike Wong, Santiago Segarra, Sathiy Kumar Mani, Trevor Eberl, Anatoliy Panasyuk, Ravi Netravali, Ranveer Chandra, and Srikanth Kandula. 2024. NetVigil: Robust and Low-Cost Anomaly Detection for East-West Data Center Security. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*. 1771–1789.
- [92] Peiwei Hu, Ruigang Liang, and Kai Chen. 2024. DeGPT: Optimizing Decompiler Output with LLM. In *Proceedings of the Network and Distributed System Security Symposium*.
- [93] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt Leaking Attacks Against Large Language Model Applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3600–3614.
- [94] Yintong Huo, Yichen Li, Yuxin Su, Pinjia He, Zifan Xie, and Michael R Lyu. 2023. AutoLog: A Log Sequence Synthesis Framework for Anomaly Detection. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 497–509.
- [95] IEEE. 2000. *IEEE Xplore Digital Library*. <https://ieeexplore.ieee.org>
- [96] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2023. SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 2620–2638. <https://doi.org/10.1109/SP46215.2023.10179405>
- [97] Muhammad Adil Inam, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2022. FAuST: Striking A Bargain between Forensic Auditing’s Security and Throughput. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 813–826.
- [98] Yang Ji, Sangho Lee, Evan Downing, Weiren Wang, Mattia Fazzini, Taesoo Kim, Alessandro Orso, and Wenke Lee. 2017. Rain: Refinable Attack Investigation with On-demand Inter-Process Information Flow Tracking. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 377–390.
- [99] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning. In *Proceedings of the 33rd USENIX Security Symposium*. 5197–5214.
- [100] Baoxiang Jiang, T Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. 2025. Orthrus: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In *Proceedings of the USENIX Security Symposium*.
- [101] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 2016. Exploring the Limits of Language Modeling. *arXiv preprint arXiv:1602.02410* (2016).
- [102] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361* (2020).
- [103] Alexander D. Kent. 2015. Comprehensive, Multi-Source Cyber-Security Events. Los Alamos National Laboratory. <https://doi.org/10.17021/1179829>
- [104] LG Kersta, PD Bricker, and EE David Jr. 1960. Human or Machine?—A Study of Voice Naturalness. *The Journal of the Acoustical Society of America* 32, 11_Supplement (1960), 1502–1502.
- [105] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, and Joarder Kamruzzaman. 2019. Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges. *Cybersecurity* 2, 1 (2019), 1–22.
- [106] Aaron Kili. [n. d.]. Sysdig—A Powerful System Monitoring and Troubleshooting Tool for Linux.
- [107] Sunnie SY Kim, Q Vera Liao, Mihaela Vorvoreanu, Stephanie Ballard, and Jennifer Wortman Vaughan. 2024. “I’m Not Sure, But...”: Examining the Impact of Large Language Models’ Uncertainty Expression on User Reliance and Trust. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. 822–835.
- [108] Isaiah J King and H Howie Huang. 2023. Euler: Detecting Network Lateral Movement via Scalable Temporal Link Prediction. *ACM Transactions on Privacy and Security* 26, 3 (2023), 1–36.
- [109] Thomas N Kipf and Max Welling. 2016. Variational Graph Auto-Encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [110] Eric D Knapp. 2024. *Industrial Network Security: Securing Critical Infrastructure Networks for Smart Grid, SCADA, and other Industrial Control Systems*. Elsevier.
- [111] Yonghwi Kwon, Fei Wang, Weihang Wang, Kyu Hyung Lee, Wen-Chuan Lee, Shiqing Ma, Xiangyu Zhang, Dongyan Xu, Somesh Jha, Gabriela Ciocarlie, et al. 2018. MCI: Modeling-based Causality Inference in Audit Logging for Attack Investigation. In *Proceedings of the Network and Distributed Systems Security Symposium*.
- [112] Grafana Labs. 2014. *Grafana: The Open Observability Platform*. <https://grafana.com/>
- [113] Van-Hoang Le and Hongyu Zhang. 2021. Log-Based Anomaly Detection without Log Parsing. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 492–504.

- [114] Van-Hoang Le and Hongyu Zhang. 2023. Log Parsing with Prompt-Based Few-Shot Learning. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 2438–2449.
- [115] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. High Accuracy Attack Provenance via Binary-based Execution Partition. In *Proceedings of the Network and Distributed System Security Symposium*, Vol. 16.
- [116] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. LogGC: Garbage Collecting Audit Log. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*. 1005–1016.
- [117] Huanruo Li, Yunfei Guo, Shumin Huo, Hongchao Hu, and Penghao Sun. 2022. Defensive Deception Framework Against Reconnaissance Attacks in the Cloud with Deep Reinforcement Learning. *Science China Information Sciences* 65, 7 (2022), 170305.
- [118] Jiawei Li, Ru Zhang, and Jianyi Liu. 2023. ConLBS: An Attack Investigation Approach Using Contrastive Learning with Behavior Sequence. *Sensors* 23, 24 (2023), 9881.
- [119] Jiawei Li, Ru Zhang, and Jianyi Liu. 2023. ProvGRP: A Context-Aware Provenance Graph Reduction and Partition Approach for Facilitating Attack Investigation. *Electronics* 13, 1 (2023), 100.
- [120] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2024. NodLink: An Online System for Fine-Grained APT Attack Detection and Investigation. In *Proceedings of the Network and Distributed System Security Symposium*.
- [121] Teng Li, Jianfeng Ma, and Cong Sun. 2017. NetPro: Detecting Attacks in MANET Routing with Provenance and Verification. *Science China Information Sciences* 60, 11 (2017), 118101.
- [122] Xiaoxiang Li, Xinyu Jiang, Hai Wan, and Xinbin Zhao. 2025. TeRed: Normal Behavior-Based Efficient Provenance Graph Reduction for Large-Scale Attack Forensics. *IEEE Transactions on Information Forensics and Security* (2025).
- [123] Xiaoyun Li, Hongyu Zhang, Van-Hoang Le, and Pengfei Chen. 2024. LogShrink: Effective Log Compression by Leveraging Commonality and Variability of Log Data. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [124] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-Level Code Generation with Alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [125] Yanjie Li, Zhen Xiang, Nathaniel D Bastian, Dawn Song, and Bo Li. 2024. IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks. In *Proceedings of the NeurIPS 2024 Workshop on Open-World Agents*.
- [126] Yuanlin Li, Zhiwei Xu, Min Zhou, Hai Wan, and Xibin Zhao. 2024. Trident: Detecting SQL Injection Attacks via Abstract Syntax Tree-based Neural Network. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2225–2229.
- [127] Zhenyuan Li, Qi Alfred Chen, Runqing Yang, Yan Chen, and Wei Ruan. 2021. Threat Detection and Investigation with System-Level Provenance Graphs: A Survey. *Computer and Security* 106, C (jul 2021), 16 pages. <https://doi.org/10.1016/j.cose.2021.102282>
- [128] Hung-Jen Liao, Chun-Hung Richard Lin, Ying-Chih Lin, and Kuang-Yuan Tung. 2013. Intrusion Detection System: A Comprehensive Review. *Journal of Network and Computer Applications* 36, 1 (2013), 16–24.
- [129] Soo Yee Lim, Bogdan Stelea, Xueyuan Han, and Thomas Pasquier. 2021. Secure Namespaced Kernel Audit for Containers. In *Proceedings of the ACM Symposium on Cloud Computing*. 518–532.
- [130] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuwei Chen. 2016. Log Clustering Based Problem Identification for Online Service Systems. In *Proceedings of the International Conference on Software Engineering Companion*. 102–111.
- [131] Brian Lindauer. 2020. Insider Threat Test Dataset. (9 2020). <https://doi.org/10.1184/R1/12841247.v1>
- [132] Guangrui Liu, Weizhe Zhang, Xinjie Li, Kaisheng Fan, and Shui Yu. 2022. VulnerGAN: A Backdoor Attack through Vulnerability Amplification against Machine Learning-Based Network Intrusion Detection Systems. *Science China Information Sciences* 65, 7 (2022), 170303.
- [133] Jason Liu, Muhammad Adil Inam, Akul Goyal, Andy Riddle, Kim Westfall, and Adam Bates. 2025. What We Talk About When We Talk About Logs: Understanding the Effects of Dataset Quality on Endpoint Threat Detection Research. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy*. IEEE, 112–129.
- [134] Jian Liu, Junjie Yan, Zhengwei Jiang, Xuren Wang, and Jun Jiang. 2022. A Graph Learning Approach with Audit Records for Advanced Attack Investigation. In *Proceedings of the IEEE Global Communications Conference*. IEEE, 897–902.
- [135] Jinyang Liu, Jieming Zhu, Shilin He, Pinjia He, Zibin Zheng, and Michael R Lyu. 2019. Logzip: Extracting Hidden Structures via Iterative Clustering for Log Compression. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 863–873.
- [136] Shuai Liu, Yiheng Pan, Kun Hong, Ruite Fei, Chenhao Lin, Qian Li, and Chao Shen. 2025. Backdoor Threats in Large Language Models—A Survey. *Science China Information Sciences* 68, 9 (2025), 1–34.

- [137] Yudong Liu, Xu Zhang, Shilin He, Hongyu Zhang, Liquan Li, Yu Kang, Yong Xu, Minghua Ma, Qingwei Lin, Yingnong Dang, et al. 2022. UniParser: A Unified Log Parser for Heterogeneous Log Data. In *Proceedings of the ACM Web Conference*. 1893–1901.
- [138] Scott Lupton, Hironori Washizaki, Nobukazu Yoshioka, and Yoshiaki Fukazawa. 2021. Literature Review on Log Anomaly Detection Approaches Utilizing Online Parsing Methodology. In *Proceedings of the 2021 28th Asia-Pacific Software Engineering Conference*. 559–563. <https://doi.org/10.1109/APSEC53868.2021.00068>
- [139] Mingqi Lv, HongZhe Gao, Xuebo Qiu, Tieming Chen, Tiantian Zhu, Jinyin Chen, and Shouling Ji. 2024. TREC: APT Tactic/Technique Recognition via Few-Shot Provenance Subgraph Learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 139–152.
- [140] Yang Lv, Shaona Qin, Zifeng Zhu, Zhuocheng Yu, Shudong Li, and Weihong Han. 2022. A Review of Provenance Graph based APT Attack Detection: Applications and Developments. In *Proceedings of the 2022 7th IEEE International Conference on Data Science in Cyberspace*. 498–505. <https://doi.org/10.1109/DSC55868.2022.00075>
- [141] Shiqing Ma, Juan Zhai, Yonghui Kwon, Kyu Hyung Lee, Xiangyu Zhang, Gabriela Ciocarlie, Ashish Gehani, Vinod Yegneswaran, Dongyan Xu, and Somesh Jha. 2018. Kernel-Supported Cost-Effective Audit Logging for Causality Tracking. In *Proceedings of the 2018 USENIX Annual Technical Conference*. 241–254.
- [142] Shiqing Ma, Juan Zhai, Fei Wang, Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2017. MPI: Multiple Perspective Attack Investigation with Semantic Aware Execution Partitioning. In *Proceedings of the 26th USENIX Security Symposium*. 1111–1128.
- [143] Shiqing Ma, Xiangyu Zhang, and Dongyan Xu. 2016. ProTracer: Towards Practical Provenance Tracing by Alternating between Logging and Tainting. In *Proceedings of the 23rd Annual Network and Distributed System Security Symposium*.
- [144] Pedro Manso, José Moura, and Carlos Serrão. 2019. SDN-Based Intrusion Detection System for Early Detection and Mitigation of DDoS Attacks. *Information* 10, 3 (2019), 106.
- [145] Emaad Manzoor, Sadegh M Milajerdi, and Leman Akoglu. 2016. Fast Memory-Efficient Anomaly Detection in Streaming Heterogeneous Graphs. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1035–1044.
- [146] Qinghua Mao, Xi Lin, Wenchao Xu, Yuxin Qi, Xiu Su, Gaolei Li, and Jianhua Li. 2025. FeCoGraph: Label-Aware Federated Graph Contrastive Learning for Few-Shot Network Intrusion Detection. *IEEE Transactions on Information Forensics and Security* (2025).
- [147] Yuyi Mao, Changsheng You, Jun Zhang, Kaibin Huang, and Khaled B Letaief. 2017. A Survey on Mobile Edge Computing: The Communication Perspective. *IEEE Communications Surveys and Tutorials* 19, 4 (2017), 2322–2358.
- [148] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building A Large Annotated Corpus of English: The Penn Treebank. *Computational Linguistics* 19, 2 (1993), 313–330.
- [149] Ariana Martino, Michael Iannelli, and Coleen Truong. 2023. Knowledge Injection to Counter Large Language Model (LLM) Hallucination. In *European Semantic Web Conference*. Springer, 182–185.
- [150] Ines Martins, Joao S Resende, Patricia R Sousa, Simao Silva, Luis Antunes, and Joao Gama. 2022. Host-based IDS: A Review and Open Issues of An Anomaly Detection System in IoT. *Future Generation Computer Systems* 133 (2022), 95–113.
- [151] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the International Joint Conference on Artificial Intelligence*, Vol. 19. 4739–4745.
- [152] Noor Michael, Jaron Mink, Jason Liu, Sneha Gaur, Wajih Ul Hassan, and Adam Bates. 2020. On the Forensic Validity of Approximated Audit Logs. In *Proceedings of the 36th Annual Computer Security Applications Conference*. 189–202.
- [153] Microsoft. [n. d.]. Event Tracing - Win32 apps. <https://learn.microsoft.com/en-us/windows/win32/etw/event-tracing-portal>. 2020.
- [154] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781* (2013).
- [155] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed Representations of Words and Phrases and Their Compositionality. *Advances in Neural Information Processing Systems* 26 (2013).
- [156] Sadegh M Milajerdi, Birhanu Eshete, Rigel Gjomemo, and VN Venkatakrishnan. 2019. Poirot: Aligning Attack Behavior with Kernel Audit Records for Cyber Threat Hunting. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1795–1812.
- [157] Sadegh M Milajerdi, Rigel Gjomemo, Birhanu Eshete, Ramachandran Sekar, and VN Venkatakrishnan. 2019. Holmes: Real-time APT Detection through Correlation of Suspicious Information Flows. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy*. IEEE, 1137–1152.
- [158] Seyed Mohammad Mehdi Mirnajafizadeh, Ashwin Raam Sethuram, David Mohaisen, DaeHun Nyang, and Rhongho Jang. 2024. Enhancing Network Attack Detection with Distributed and In-Network Data Collection System. In *Proceedings of the 33rd USENIX Security Symposium*. 5161–5178.

- [159] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. *Proceedings of the Network and Distributed Systems Security Symposium* (2018).
- [160] Kunal Mukherjee and Murat Kantarcioglu. 2025. LLM-driven Provenance Forensics for Threat Investigation and Detection. *arXiv preprint arXiv:2508.21323* (2025).
- [161] Kunal Mukherjee, Joshua Wiedemeier, Tianhao Wang, James Wei, Feng Chen, Muhyun Kim, Murat Kantarcioglu, and Kangkook Jee. 2023. Evading Provenance-Based ML Detectors with Adversarial System Actions. In *Proceedings of the 32nd USENIX Security Symposium*. 1199–1216.
- [162] Muhammad Hassan Nasir, Salman A Khan, Muhammad Mubashir Khan, and Mahawish Fatima. 2022. Swarm Intelligence Inspired Intrusion Detection Systems—A Systematic Literature Review. *Computer Networks* 205 (2022), 108708.
- [163] Mostafa Nassar, Nirmeen A El-Bahnasawy, HossamEl-Din H Ahmed, Adel A Saleeb, and Fathi E Abd El-Samie. 2019. Network Intrusion Detection, Literature Review and Some Techniques Comparision. In *Proceedings of the 2019 15th International Computer Engineering Conference*. IEEE, 62–71.
- [164] Alexander Tobias Neumann, Yue Yin, Sulayman Sowe, Stefan Decker, and Matthias Jarke. 2024. An LLM-Driven Chatbot in Higher Education for Databases and Information Systems. *IEEE Transactions on Education* (2024).
- [165] Zhibin Ni, Pan Fan, Shengzhuo Dai, Bo Zhang, Hai Wan, and Xibin Zhao. 2025. FG-CIBGC: A Unified Framework for Fine-Grained and Class-Incremental Behavior Graph Classification. In *Proceedings of the Web Conference*.
- [166] Weina Niu, Zhenqi Yu, Zimu Li, Beibei Li, Runzi Zhang, and Xiaosong Zhang. 2022. LogTracer: Efficient Anomaly Tracing Combining System Log Detection and Provenance Graph. In *Proceedings of the IEEE Global Communications Conference*. IEEE, 3356–3361.
- [167] Christine Nussbaum, Sascha Frühholz, and Stefan R Schweinberger. 2025. Understanding Voice Naturalness. *Trends in Cognitive Sciences* (2025).
- [168] Connected Papers. 2020. *Connected Papers: A Visual Tool for Researchers*. <https://www.connectedpapers.com>
- [169] Nohil Park, Heeseung Kim, Che Hyun Lee, Jooyoung Choi, Jiheum Yeom, and Sungroh Yoon. 2025. NanoVoice: Efficient Speaker-Adaptive Text-to-Speech for Multiple Speakers. In *Proceedings of the ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 1–5.
- [170] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eysers, Margo Seltzer, and Jean Bacon. 2017. Practical Whole-System Provenance Capture. In *Proceedings of the 2017 Symposium on Cloud Computing*. 405–418.
- [171] Igor Pavlov. 2001. LZMA SDK (Software Development Kit). <https://www.7-zip.org/>
- [172] Cheng Peng, Xi Yang, Aokun Chen, Kaleb E Smith, Nima PourNejatian, Anthony B Costa, Cheryl Martin, Mona G Flores, Ying Zhang, Tanja Magoc, et al. 2023. A Study of Generative Large Language Model For Medical Research and Healthcare. *NPJ Digital Medicine* 6, 1 (2023), 210.
- [173] Yihao Peng, Tongxin Zhang, Jieshao Lai, Yuxuan Zhang, Yiming Wu, Hai Wan, and Xibin Zhao. 2025. AutoLabel: Automated Fine-Grained Log Labeling for Cyber Attack Dataset Generation. In *34th USENIX Security Symposium (USENIX Security 25)*. 547–566.
- [174] Prometheus. 2014. *Prometheus - Monitoring System & Time Series Database*. <https://prometheus.io/>
- [175] Jiaying Qi, Zhongzhi Luan, Shaohan Huang, Carol Fung, Hailong Yang, and Depei Qian. 2023. SpikeLog: Log-based Anomaly Detection via Potential-Assisted Spiking Neuron Network. *IEEE Transactions on Knowledge and Data Engineering* 36, 12 (2023), 9322–9335.
- [176] Wei Qiao, Yebo Feng, Teng Li, Zhuo Ma, Yulong Shen, JianFeng Ma, and Yang Liu. 2025. Slot: Provenance-Driven APT Detection through Graph Reinforcement Learning. In *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security*.
- [177] QuickLZ. 2006. *QuickLZ: Fastest Compression Library*. <http://www.quicklz.com/>
- [178] Alec Radford. 2018. Improving Language Understanding by Generative Pre-Training. (2018).
- [179] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the Limits of Transfer Learning with A Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.
- [180] Baishakhi Ray, Vincent Hellendoorn, Saheel Godhane, Zhaopeng Tu, Alberto Bacchelli, and Premkumar Devanbu. 2016. On the "Naturalness" of Buggy Code. In *Proceedings of the 38th International Conference on Software Engineering*. 428–439.
- [181] Bace Rebecca and Peter Mell. 2001. Intrusion Detection Systems. *National Institute of Standards and Technology, Special Publication* (2001).
- [182] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. FLASH: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society, 139–139.
- [183] Yi Ren, Yangjun Ruan, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. 2019. FastSpeech: Fast, Robust and Controllable Text to Speech. *Advances in Neural Information Processing Systems* 32 (2019).

- [184] Andy Riddle, Kim Westfall, and Adam Bates. 2023. Atlasv2: Atlas attack engagements, version 2. *arXiv preprint arXiv:2401.01341* (2023).
- [185] Malajah Roberts, Jonathan Anderson, William Delgado, Richard Johnson, and Lawrence Spencer. 2024. Extending Contextual Length and World Knowledge Generalization in Large Language Models. (2024).
- [186] Kirk Rodrigues, Yu Luo, and Ding Yuan. 2021. CLP: Efficient and Scalable Search on Compressed Text Logs. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*. 183–198.
- [187] Ronald Rosenfeld. 2000. Two Decades of Statistical Language Modeling: Where Do We Go from Here? *Proceedings of the IEEE* 88, 8 (2000), 1270–1278.
- [188] Tejaswini S and Azra Nasreen. 2021. Survey on Online Log Parsers. *Regular issue* (2021). <https://api.semanticscholar.org/CorpusID:236861650>
- [189] Vijay Samuel. 2018. Monitoring Anything and Everything with Beats at eBay.(2018). (2018).
- [190] Michael Schindler. 1999. *SZIP Compression*. <http://www.compressconsult.com/szip/>
- [191] Frank Schwellinger. 2008. *Ocamyd: A File (De-)Compressor Based on the DMC Algorithm*. <https://www.geocities.ws/ocamyd/>
- [192] Issam Sedki, Abdelwahab Hamou-Lhadj, Otmane Ait-Mohamed, and Mohammed A Shehab. 2022. An Effective Approach for Parsing Large Log Files. In *Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 1–12.
- [193] R Sekar, Hanke Kimm, and Rohit Aich. 2024. eAudit: A Fast, Scalable and Deployable Audit Data Collection System. In *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 3571–3589.
- [194] Julian Seward. 1996. *bzip2: A High-Quality Data Compressor*. <http://www.bzip.org/>
- [195] Claude E Shannon. 1948. A Mathematical Theory of Communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423.
- [196] Claude E Shannon. 1951. The Redundancy of English. In *Cybernetics; Transactions of the 7th Conference, New York: Josiah Macy, Jr. Foundation*. 248–272.
- [197] Madhukar Shrestha, Yonghyun Kim, Jeehyun Oh, Junghwan Rhee, Yung Ryn Choe, Fei Zuo, Myungah Park, and Gang Qian. 2023. ProvSec: Open Cybersecurity System Provenance Analysis Benchmark Dataset with Labels. *International Journal of Networked and Distributed Computing* 11, 2 (2023), 112–123.
- [198] Rakesh Shrestha, Atefeh Omidkar, Sajjad Ahmadi Roudi, Robert Abbas, and Shiho Kim. 2021. Machine-Learning-Enabled Intrusion Detection System for Cellular Connected UAV Networks. *Electronics* 10, 13 (2021), 1549.
- [199] Zhuoxue Song, Ziming Zhao, Fan Zhang, Gang Xiong, Guang Cheng, Xinjie Zhao, Shize Guo, and Binbin Chen. 2022. I²RNN: An Incremental and Interpretable Recurrent Neural Network for Encrypted Traffic Classification. *IEEE Transactions on Dependable and Secure Computing* (2022).
- [200] Manolis Stamatogiannakis, Paul Groth, and Herbert Bos. 2015. Looking Inside the Black-Box: Capturing Data Provenance Using Dynamic Instrumentation. In *Provenance and Annotation of Data and Processes: 5th International Provenance and Annotation Workshop, IPAW 2014, Cologne, Germany, June 9-13, 2014. Revised Selected Papers 5*. Springer, 155–167.
- [201] Branka Stojanović, Katharina Hofer-Schmitz, and Ulrike Kleb. 2020. APT Datasets and Attack Modeling for Automated Detection Methods: A Review. *Computer Security* 92 (2020), 101734. <https://api.semanticscholar.org/CorpusID:213320542>
- [202] Hongbin Sun, Su Wang, Zhiliang Wang, Zheyu Jiang, Dongqi Han, and Jiahai Yang. 2024. AudiTrim: A Real-time, General, Efficient, and Low-overhead Data Compaction System for Intrusion Detection. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. 263–277.
- [203] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. Intellicode Compose: Code Generation Using Transformer. In *Proceedings of the 28th ACM joint meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1433–1443.
- [204] Dan Tang, Yudong Yan, Chenjun Gao, Wei Liang, and Wenqiang Jin. 2023. LtRFT: Mitigate the Low-Rate Data Plane DDoS Attack with Learning-to-Rank Enabled Flow Tables. *IEEE Transactions on Information Forensics and Security* 18 (2023), 3143–3157.
- [205] Yutao Tang, Ding Li, Zhichun Li, Mu Zhang, Kangkook Jee, Xusheng Xiao, Zhenyu Wu, Junghwan Rhee, Fengyuan Xu, and Qun Li. 2018. NodeMerge: Template Based Efficient Data Reduction for Big-Data Causality Analysis. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1324–1337.
- [206] Joerg Thalheim, Pramod Bhatotia, and Christof Fetzer. 2016. Inspector: Data Provenance Using Intel Processor Trace (PT). In *Proceedings of the 2016 IEEE 36th International Conference on Distributed Computing Systems*. IEEE, 25–34.
- [207] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language Models for Dialog Applications. *arXiv preprint arXiv:2201.08239* (2022).
- [208] ThoughtWorks. 2004. *Selenium RC*. <http://www.seleniumhq.org/projects/remote-control/>

- [209] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).
- [210] Aqua Tracee. 2022. Runtime eBPF Threat Detection Engine.
- [211] Devharsh Trivedi, Aymen Boudguiga, Nesrine Kaaniche, and Nikos Triandopoulos. 2023. SigML++: Supervised Log Anomaly with Probabilistic Polynomial Approximation. *Cryptography* 7, 4 (2023), 52.
- [212] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. *Advances in Neural Information Processing Systems* 30 (2017).
- [213] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph Attention Networks. *stat* 1050, 20 (2017), 10–48550.
- [214] Arthur Vervaeke, Raja Chiky, and Mar Callau-Zori. 2021. USTEP: Unfixed Search Tree for Efficient Log Parsing. In *Proceedings of the 2021 IEEE International Conference on Data Mining*. IEEE, 659–668.
- [215] David Wagner and Paolo Soto. 2002. Mimicry Attacks on Host-Based Intrusion Detection Systems. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*. 255–264.
- [216] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A Gunter, et al. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In *Proceedings of the Network and Distributed System Security Symposium*.
- [217] Rui Wang, Devin Gibson, Kirk Rodrigues, Yu Luo, Yun Zhang, Kaibo Wang, Yupeng Fu, Ting Chen, and Ding Yuan. 2024. μ Slope: High Compression and Fast Search on Semi-Structured Logs. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 529–544.
- [218] Ruihua Wang, Yihao Peng, Yilun Sun, Xuancheng Zhang, Hai Wan, and Xibin Zhao. 2023. TeSec: Accurate Server-Side Attack Investigation for Web Applications. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. IEEE, 2799–2816.
- [219] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. 2022. threaTrace: Detecting and Tracing Host-Based Threats in Node Level Through Provenance Graph Learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3972–3987.
- [220] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent Abilities of Large Language Models. *arXiv preprint arXiv:2206.07682* (2022).
- [221] Wei Wei, Sijin Chen, Cen Chen, Heshi Wang, Jing Liu, Zhongyao Cheng, and Xiaofeng Zou. 2024. HEN: A Novel Hybrid Explainable Neural Network Based Framework for Robust Network Intrusion Detection. *Science China Information Sciences* 67, 7 (2024), 170304.
- [222] Cong Wu, Jianfei Sun, Jing Chen, Mamoun Alazab, Yang Liu, and Yang Xiang. 2025. TCG-IDS: Robust Network Intrusion Detection via Temporal Contrastive Graph Learning. *IEEE Transactions on Information Forensics and Security* (2025).
- [223] Weiheng Wu, Wei Qiao, Teng Li, Yebo Feng, Zhuo Ma, Jianfeng Ma, and Yang Liu. 2025. ProvX: Generating Counterfactual-Driven Attack Explanations for Provenance-Based Detection. *arXiv preprint arXiv:2508.06073* (2025).
- [224] Yafeng Wu, Yulai Xie, Xuelong Liao, Pan Zhou, Dan Feng, Lin Wu, Xuan Li, Avani Wildani, and Darrell Long. 2022. Paradise: Real-Time, Generalized, and Distributed Provenance-Based Intrusion Detection. *IEEE Transactions on Dependable and Secure Computing* 20, 2 (2022), 1624–1640.
- [225] Yixuan Wu, Long Zhang, Lin Yang, Feng Yang, Linru Ma, Zhoumin Lu, and Wen Jiang. 2025. Intrusion Detection for Internet of Things: An Anchor Graph Clustering Approach. *IEEE Transactions on Information Forensics and Security* (2025).
- [226] Tong Xiao, Zhe Quan, Zhi-Jie Wang, Kaiqi Zhao, Xiangke Liao, Huang Huang, Yunfei Du, and Kenli Li. 2023. LPV: A Log Parsing Framework Based on Vectorization. *IEEE Transactions on Network and Service Management* 20, 3 (2023), 2711–2725.
- [227] Yulai Xie, Dan Feng, Yuchong Hu, Yan Li, Staunton Sample, and Darrell Long. 2018. Pagoda: A Hybrid Approach to Enable Efficient Real-Time Provenance Based Intrusion Detection in Big Data Environments. *IEEE Transactions on Dependable and Secure Computing* 17, 6 (2018), 1283–1296.
- [228] Yulai Xie, Kiran-Kumar Muniswamy-Reddy, Darrell DE Long, Ahmed Amer, Dan Feng, and Zhipeng Tan. 2011. Compressing Provenance Graphs. In *Proceedings of the 3rd USENIX Workshop on the Theory and Practice of Provenance*.
- [229] Junjielong Xu, Qiulai Fu, Zhouxiang Zhu, Yutong Cheng, Zhijing Li, Yuchi Ma, and Pinjia He. 2023. Hue: A User-Adaptive Parser for Hybrid Logs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 413–424.
- [230] Jiachen Xu, Xiaokui Shu, and Zhou Li. 2024. Understanding and Bridging the Gap between Unsupervised Network Representation Learning and Security Analytics. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy*. IEEE, 3590–3608.

- [231] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I Jordan. 2009. Detecting Large-scale System Problems by Mining Console Logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*. 117–132.
- [232] Zhiqiang Xu, Pengcheng Fang, Changlin Liu, Xusheng Xiao, Yu Wen, and Dan Meng. 2022. DepComm: Graph Summarization on System Audit Logs for Attack Investigation. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. IEEE, 540–557.
- [233] Zhiwei Xu, Shaohua Qiang, Dinghong Song, Min Zhou, Hai Wan, Xibin Zhao, Ping Luo, and Hongyu Zhang. 2024. DSFM: Enhancing Functional Code Clone Detection with Deep Subtree Interactions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [234] Zhang Xu, Zhenyu Wu, Zhichun Li, Kangkook Jee, Junghwan Rhee, Xusheng Xiao, Fengyuan Xu, Haining Wang, and Guofei Jiang. 2016. High Fidelity Data Reduction for Big Data Security Dependency Analyses. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 504–516.
- [235] Zhiwei Xu, Min Zhou, Xibin Zhao, Yang Chen, Xi Cheng, and Hongyu Zhang. 2023. xASTNN: Improved Code Representations for Industrial Practice. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1727–1738.
- [236] Yu Xue, Bernard-marie Onzo, and Ferrante Neri. 2021. Intrusion Detection System Based on an Updated ANN Model. In *Advances in Swarm Intelligence: 12th International Conference, ICSI 2021, Qingdao, China, July 17–21, 2021, Proceedings, Part II* 12. Springer, 472–479.
- [237] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. ProGrapher: An Anomaly Detection System based on Provenance Graph Embedding. In *Proceedings of the 32nd USENIX Security Symposium*. 4355–4372.
- [238] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Semi-Supervised Log-Based Anomaly Detection via Probabilistic Label Estimation. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1448–1460.
- [239] Runqing Yang, Shiqing Ma, Haitao Xu, Xiangyu Zhang, and Yan Chen. 2020. UIScope: Accurate, Instrumentation-free, and Visible Attack Investigation for GUI Applications. In *Proceedings of the Network and Distributed Systems Security Symposium*.
- [240] Zhaohui Yang, Wei Xu, Le Liang, Yuanhao Cui, Zhijin Qin, and Mérouane Debbah. 2025. On Privacy, Security, and Trustworthiness in Distributed Wireless Large AI Models. *Science China Information Sciences* 68, 7 (2025), 1–15.
- [241] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. 2023. LLM Lies: Hallucinations are not Bugs, but Features as Adversarial Examples. *arXiv preprint arXiv:2310.01469* (2023).
- [242] Kundi Yao, Heng Li, Weiyl Shang, and Ahmed E Hassan. 2020. A Study of the Performance of General Compressors on Log Files. *Empirical Software Engineering* 25 (2020), 3043–3085.
- [243] Kundi Yao, Mohammed Sayagh, Weiyl Shang, and Ahmed E Hassan. 2021. Improving State-of-the-Art Compression Techniques for Log Management Tools. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2748–2760.
- [244] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A Survey on Large Language Model (LLM) Security and Privacy: The Good, the Bad, and the Ugly. *High-Confidence Computing* (2024), 100211.
- [245] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-Wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. 116–127.
- [246] Kun Yin, Meng Yan, Ling Xu, Zhou Xu, Zhao Li, Dan Yang, and Xiaohong Zhang. 2020. Improving Log-Based Anomaly Detection with Component-Aware Analysis. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 667–671.
- [247] Guangba Yu, Pengfei Chen, Pairui Li, Tianjun Weng, Haibing Zheng, Yuetang Deng, and Zibin Zheng. 2023. LogReducer: Identify and Reduce Log Hotspots in Kernel on the Fly. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 1763–1775.
- [248] Le Yu, Shiqing Ma, Zhuo Zhang, Guan hong Tao, Xiangyu Zhang, Dongyan Xu, Vincent E Urias, Han Wei Lin, Gabriela F Ciocarlie, Vinod Yegneswaran, et al. 2021. ALchemist: Fusing Application and Audit Logs for Precise Attack Provenance without Instrumentation. In *Proceedings of the Network and Distributed System Security Symposium*.
- [249] Siyu Yu, Yifan Wu, Ying Li, and Pinjia He. 2024. Unlocking the Power of Numbers: Log Compression via Numeric Token Parsing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 919–930.
- [250] Jun Zengy, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. 2022. ShadeWatcher: Recommendation-Guided Cyber Threat Analysis Using System Audit Records. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. IEEE, 489–506.
- [251] Chao Zha, Zhiyu Wang, Yifei Fan, Bing Bai, Yinjie Zhang, Sainan Shi, and Ruyun Zhang. 2025. A-NIDS: Adaptive Network Intrusion Detection System based on Clustering and Stacked CTGAN. *IEEE Transactions on Information Forensics and Security* (2025).

- [252] Bo Zhang, Yansong Gao, Changlong Yu, Boyu Kuang, Zhi Zhang, Hyoungshick Kim, and Anmin Fu. 2025. TAPAS: An Efficient Online APT Detection with Task-guided Process Provenance Graph Segmentation and Analysis. In *Proceedings of the USENIX Security Symposium*. 607–624.
- [253] Pei Zhang, Fangzhou He, Han Zhang, Jiankun Hu, Xiaohong Huang, Jilong Wang, Xia Yin, Huahong Zhu, and Yahui Li. 2023. Real-Time Malicious Traffic Detection with Online Isolation Forest over SD-WAN. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2076–2090.
- [254] Shenglin Zhang, Yuhe Ji, Jiaqi Luan, Xiaohui Nie, Ziang Chen, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. End-to-End Automl for Unsupervised Log Anomaly Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1680–1692.
- [255] Tianzhu Zhang, Han Qiu, Gabriele Castellano, Myriana Rifai, Chung Shue Chen, and Fabio Pianese. 2023. System Log Parsing: A Survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2023), 8596–8614. <https://doi.org/10.1109/TKDE.2022.3222417>
- [256] Tianye Zhang, Xumeng Wang, Zongzhuang Li, Fangzhou Guo, Yuxin Ma, and Wei Chen. 2017. A Survey of Network Anomaly Visualization. *Science China Information Sciences* 60, 12 (2017), 121101.
- [257] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust Log-Based Anomaly Detection on Unstable Log Data. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 807–817.
- [258] Huaqin Zhao, Zhengliang Liu, Zihao Wu, Yiwei Li, Tianze Yang, Peng Shu, Shaochen Xu, Haixing Dai, Lin Zhao, Gengchen Mai, et al. 2024. Revolutionizing Finance with LLMs: An Overview of Applications and Insights. *arXiv preprint arXiv:2401.11641* (2024).
- [259] Jianjin Zhao, Qi Li, Zewei Han, Junsong Fu, Guoshun Nan, Meng Shen, and Bharat K Bhargava. 2024. ReTrial: Robust Encrypted Malicious Traffic Detection via Discriminative Relation Incorporation and Misleading Relation Correction. *IEEE Transactions on Information Forensics and Security* (2024).
- [260] Ruijie Zhao, Xianwen Deng, Zhicong Yan, Jun Ma, Zhi Xue, and Yijun Wang. 2022. MT-FlowFormer: A Semi-Supervised Flow Transformer for Encrypted Traffic Classification. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 2576–2584.
- [261] Ying Zhao, FangFang Zhou, XiaoPing Fan, Xing Liang, and YongGang Liu. 2013. IDSRadar: A Real-Time Visualization Framework for IDS Alerts. *Science China Information Sciences* 56, 8 (2013), 1–12.
- [262] Ziming Zhao, Zhaoxuan Li, Jialun Jiang, Fengyuan Yu, Fan Zhang, Congyuan Xu, Xinjie Zhao, Rui Zhang, and Shize Guo. 2022. ERNN: Error-Resilient RNN for Encrypted Traffic Detection Towards Network-Induced Phenomena. *IEEE Transactions on Dependable and Secure Computing* (2022).
- [263] Ziming Zhao, Zhuotao Liu, Huan Chen, Fan Zhang, Zhuoxue Song, and Zhaoxuan Li. 2024. Effective DDoS Mitigation via ML-Driven In-Network Traffic Shaping. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4271–4289.
- [264] Ying Zhong, Zhiliang Wang, Xingang Shi, Jiahai Yang, and Keqin Li. 2024. RFG-HELAD: A Robust Fine-Grained Network Traffic Anomaly Detection Model Based on Heterogeneous Ensemble Learning. *IEEE Transactions on Information Forensics and Security* (2024).
- [265] Junwei Zhou, Shaowen Ying, Shulan Wang, Dongdong Zhao, Jianwen Xiang, Kaitai Liang, and Peng Liu. 2025. LogDLR: Unsupervised Cross-System Log Anomaly Detection Through Domain-Invariant Latent Representation. *IEEE Transactions on Dependable and Secure Computing* (2025).
- [266] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-Driven Log Analytics. In *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering*. IEEE, 355–366.
- [267] Tiantian Zhu, Jiayu Wang, Linqi Ruan, Chunlin Xiong, Jinkai Yu, Yaosheng Li, Yan Chen, Mingqi Lv, and Tieming Chen. 2021. General, Efficient, and Real-Time Data Compaction Strategy for APT Forensic Analysis. *IEEE Transactions on Information Forensics and Security* 16 (2021), 3312–3325.
- [268] Tiantian Zhu, Jinkai Yu, Chunlin Xiong, Wenrui Cheng, Qixuan Yuan, Jie Ying, Tieming Chen, Jiabo Zhang, Mingqi Lv, Yan Chen, et al. 2023. APTSHIELD: A Stable, Efficient and Real-time APT Detection System for Linux Hosts. *IEEE Transactions on Dependable and Secure Computing* 20, 6 (2023), 5247–5264.
- [269] Yao Zhu, Li Zhenyuan, Yangyang Wei, and Shouling Ji. 2025. The Case for Learned Provenance-based System Behavior Baseline. In *Forty-second International Conference on Machine Learning*.
- [270] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam S. Dillon. 2022. Provenance-based Intrusion Detection Systems: A Survey. *ACM Computing Surveys* 55 (2022), 1 – 36. <https://api.semanticscholar.org/CorpusID:249579087>